



数据结构

教师: 刘芳岑

邮箱: liufc@hainanu.edu.cn

学院: 计算机科学与技术学院

时间: 2026春季学期



课堂回顾



➤ 二叉排序树

BST性质

查找、插入、删除

➤ 二叉平衡树

BST性质、平衡因子的绝对值 ≤ 1

查找、插入、删除

LL、RR、LR、RL调整

➤ B树与B+树

课堂回顾

1. 对二叉排序树进行 () 遍历, 可以得到按照关键字从小到大排序的结点序列

2. 已知一组关键字为:

{4, 5, 7, 2, 1, 3, 6}

按表中的元素顺序依次插入到一棵初始为空的二叉排序树中, 画出该二叉排序树。

求在等概率的情况下查找成功的平均查找长度和查找不成功的平均查找长度。

3. 已知一组关键字为:

{4, 5, 7, 2, 1, 3, 6}

按表中的元素顺序依次插入到一棵初始为空的二叉平衡树中, 画出该二叉排序树。

求在等概率的情况下查找成功的平均查找长度和查找不成功的平均查找长度。



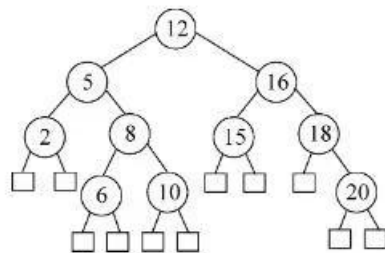
课堂回顾



4. 有一颗二叉排序树的先序遍历序列为：12 5 2 8 6 10 16 15 18 20，
回答以下问题：

- (1) 画出该二叉排序树
- (2) 给出该二叉排序树的中序遍历序列
- (3) 求在等概率情况下查找成功和不成功的平均查找长度

3. (1) 先序遍历得到的序列为(12,5,2,8,6,10,16,15,18,20)，中序序列是一个有序序列，所以为(2,5,6,8,10,12,15,16,18,20)，由先序序列和中序序列可以构造出对应的二叉树，如下图所示。



第 3 题图

(2) 中序遍历序列为(2,5,6,8,10,12,15,16,18,20)。

(3) $ASL_{成功} = (1 \times 1 + 2 \times 2 + 4 \times 3 + 3 \times 4) / 10 = 29 / 10 = 2.9$ 。

$ASL_{不成功} = (5 \times 3 + 6 \times 4) / 11 = 39 / 11$ 。



课堂回顾



➤ 哈希表的查找

装填因子：存储的记录个数/哈希表的大小= n/m 0.6~0.9

哈希函数 { 直接定址法 $h(\text{学号}) = \text{学号} - 201001001$
除留余数法 $h(k) = k \bmod p$ p 最好为质数
数字分析法

哈希冲突解决方法 { 开放定址法 (1) 线性探测法

$$d_0 = h(k)$$

$$d_i = (d_{i-1} + 1) \bmod m \quad (1 \leq i \leq m-1)$$

哈希冲突解决方法 {

(2) 平方探测法

$$d_0 = h(k)$$

$$d_i = (d_0 \pm i^2) \bmod m \quad (1 \leq i \leq m-1)$$

拉链法

ASL_{不成功} 约等于 n/m

【例9.11】 用关键字序列{7, 8, 30, 11, 18, 9, 14}构造一个哈希表，哈希表的存储空间是一个下标从0开始的一维数组，哈希函数为 $H(key)=(key \times 3) \bmod 7$ ，处理冲突采用线性探测法，要求装填（载）因子为0.7。

(1) 画出所构造的哈希表。

(2) 分别计算等概率情况下，查找成功和查找不成功的平均查找长度。

解：（1）这里 $n=7$ ， $\alpha=0.7=n/m$ ，则 $m=n/0.7=10$ 。计算各关键字存储地址的过程如下：

$$H(7)=7 \times 3 \bmod 7=0$$

$$H(8)=8 \times 3 \bmod 7=3$$

$$H(30)=30 \times 3 \bmod 7=6$$

$$H(11)=11 \times 3 \bmod 7=5$$

$$H(18)=18 \times 3 \bmod 7=5 \quad \text{冲突}$$

$$d_1=(5+1) \bmod 10=6 \quad \text{仍冲突}$$

$$d_2=(6+1) \bmod 10=7$$

$$H(9)=9 \times 3 \bmod 7=6 \quad \text{冲突}$$

$$d_1=(6+1) \bmod 10=7 \quad \text{仍冲突}$$

$$d_2=(7+1) \bmod 10=8$$

$$H(14)=14 \times 3 \bmod 7=0 \quad \text{冲突}$$

$$d_1=(0+1) \bmod 10=1$$



下标	0	1	2	3	4	5	6	7	8	9
关键字	7	14		8		11	30	18	9	
探测次数	1	2		1		1	1	3	3	

查找成功的平均查找长度计算

下标	0	1	2	3	4	5	6	7	8	9
关键字	7	14		8		11	30	18	9	
探测次数	1	2		1		1	1	3	3	

(2) 在等概率情况下:

$$ASL_{\text{成功}} = \frac{1 + 2 + 1 + 1 + 1 + 3 + 3}{7} = 1.71$$

查找不成功的平均查找长度计算

下标	0	1	2	3	4	5	6	7	8	9
关键字	7	14		8		11	30	18	9	
探测次数	1	2		1		1	1	3	3	
不成功的探测次数	3	2	1	2	1	5	4	-	-	-

(2) 在等概率情况下:

$$H(key) = (key \times 3) \bmod 7$$



任何关键字不属于{7, 8, 30, 11, 18, 9, 14}的关键字 x , $H(x)$ 在0~6之间

$$ASL_{\text{不成功}} = \frac{3 + 2 + 1 + 2 + 1 + 5 + 4}{7} = 2.57$$



对于关键字序列 (30, 15, 21, 40, 25, 26, 36, 37) , 若查找表的装填因子为 0.8, 分别采用线性探测法和拉链法解决冲突, 完成以下各题:

- (1) 设计哈希函数。
- (2) 画出哈希表。
- (3) 计算在等概率条件下查找成功和查找失败的平均查找长度。

n=8
 $\alpha=n/m=0.8$ m=10
p取小于10的素数 7
 $H(key)=key \bmod 7$

下标	0	1	2	3	4	5	6	7	8	9
关键字	21	15	30	36	25	40	26	37		
探测次数	1	1	1	3	1	1	2	6		
不成功的探测次数	9	8	7	6	5	4	3	-	-	-

$$ASL_{成功} = \frac{1 + 1 + 1 + 3 + 1 + 1 + 2 + 6}{8} = 2$$

$$ASL_{不成功} = \frac{9 + 8 + 7 + 6 + 5 + 4 + 3}{7} = 6$$

第10章 内排序

提纲 CONTENTS



1.1 排序的基本概念

1.2 插入排序

1.3 交换排序

1.4 选择排序

1.5 归并排序

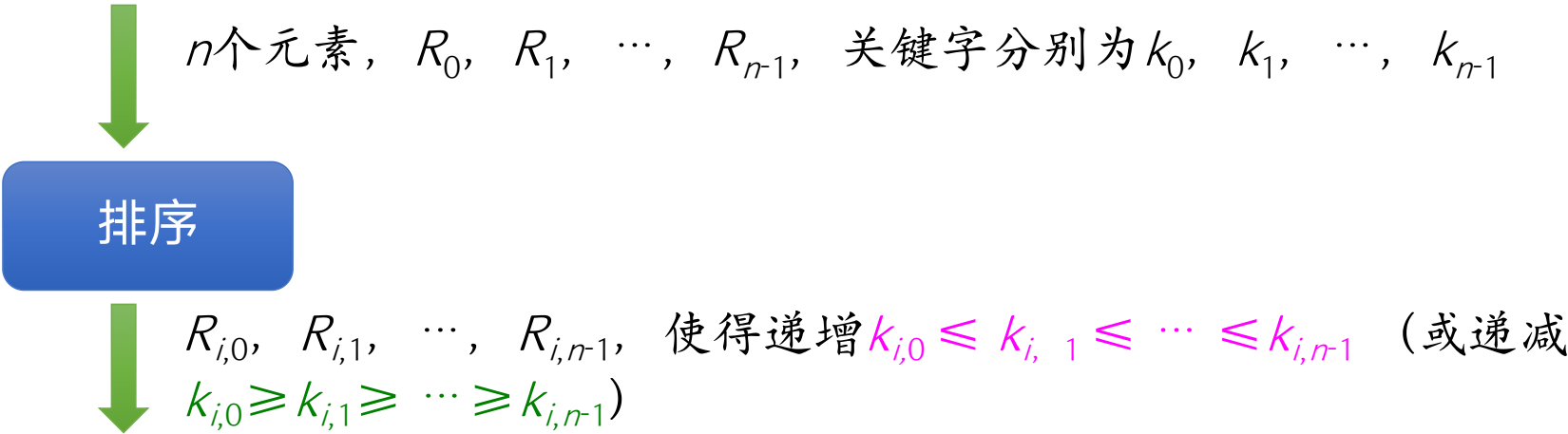
1.6 基数排序

1.7 各种内排序方法的比较和选择

1.1 排序的基本概念

1.1.1 排序的定义

所谓**排序**，是整理表中的元素，使之按关键字递增（或递减）有序排列：



说明：排序数据中可以存在相同关键字的元素。本章仅考虑**递增**排序。

1.1 排序的基本概念

1.1.2 内排序和外排序

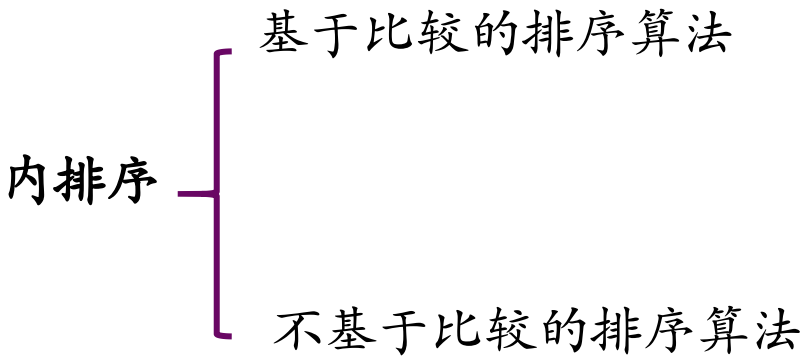
- 在排序过程中，若整个表都是放在内存中处理，排序时不涉及数据的内、外存交换，则称之为 **内排序**；
- 反之，若排序过程中要进行数据的内、外存交换，则称之为 **外排序**。



1.1 排序的基本概念

1.1.3 内排序

1. 内排序分类



- 插入排序
- 交换排序
- 选择排序
- 归并排序
- 基数排序



1.1 排序的基本概念

1.1.3 内排序

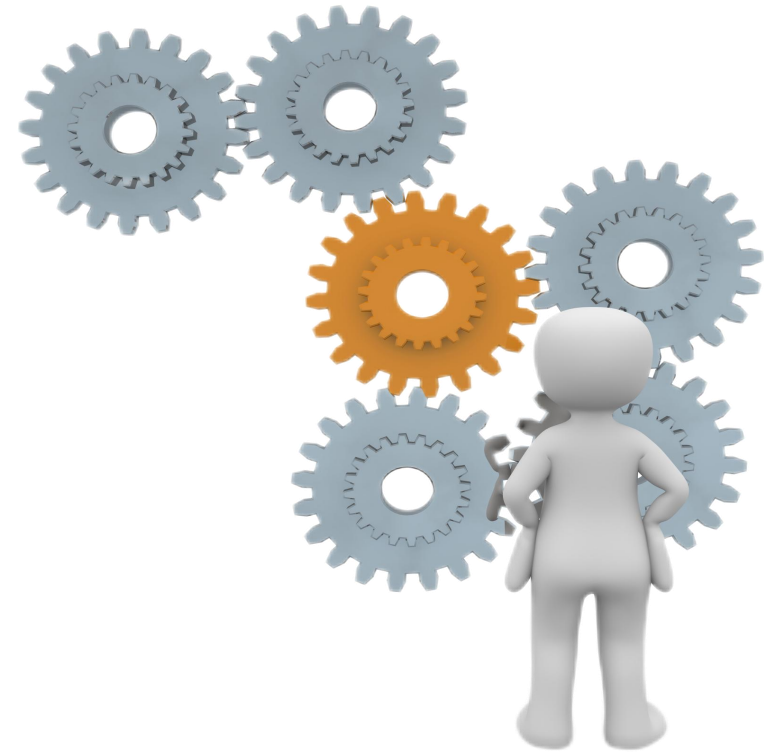
2. 基于比较的内排序

假设有3个元素(R_1, R_2, R_3), 对应的关键字为(k_1, k_2, k_3).

初始数据序列有 $3! = 6$ 种情况:

- 1, 2, 3
- 1, 3, 2
- 2, 1, 3
- 2, 3, 1
- 3, 1, 2
- 3, 2, 1

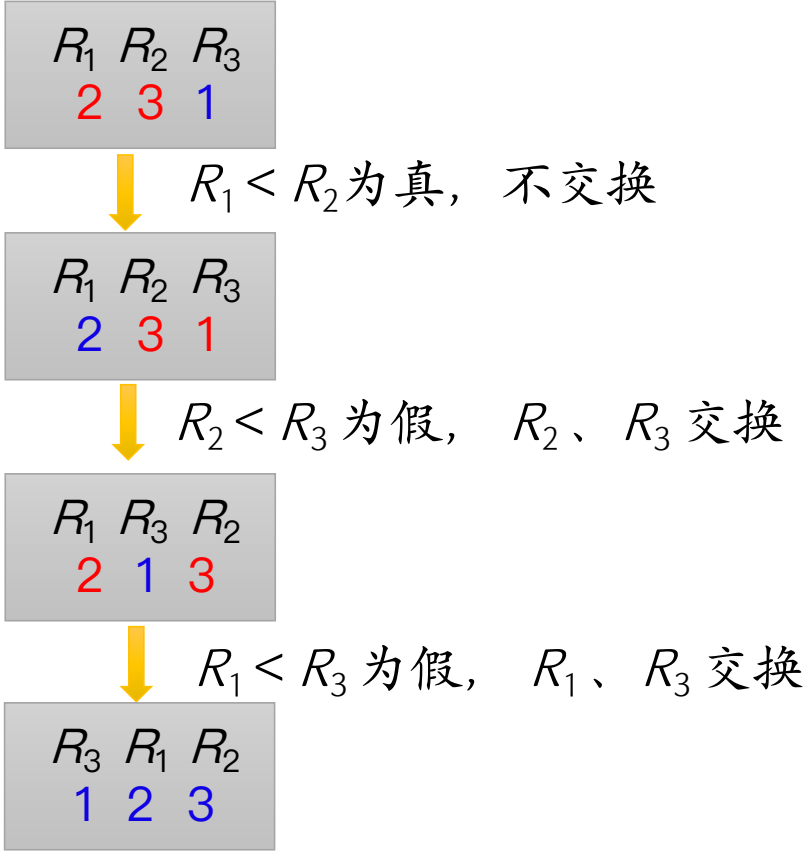
 n 个元素, 初始数据序列有 $n!$ 种情况



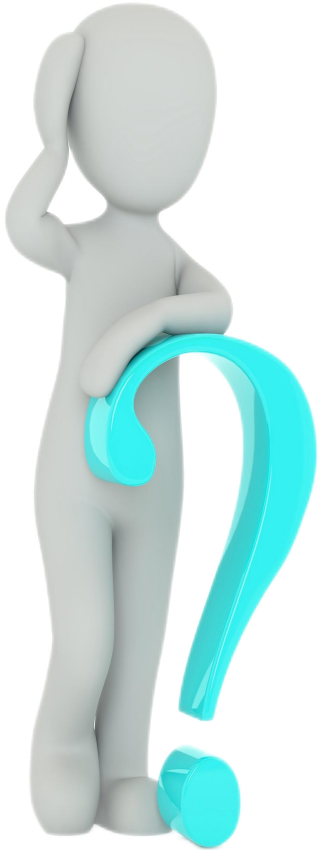
1.1 排序的基本概念

1.1.3 内排序

1 以 $(R_1, R_2, R_3) = (2, 3, 1)$ 为例，一种基于比较的排序方法：



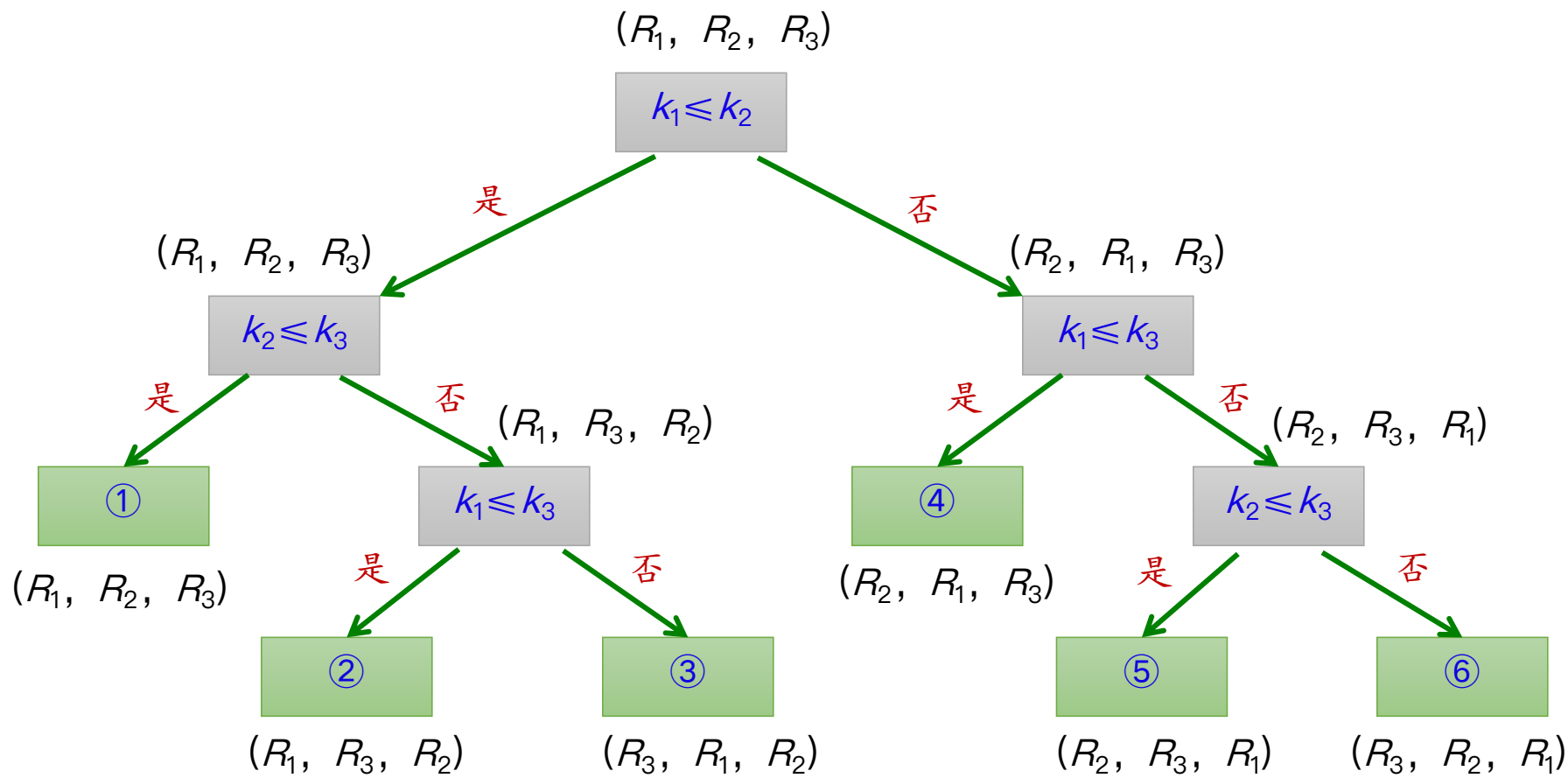
总共3次关键字比较



1.1 排序的基本概念

1.1.3 内排序

2 所有可能的初始序列的排序过程构成一个决策树：



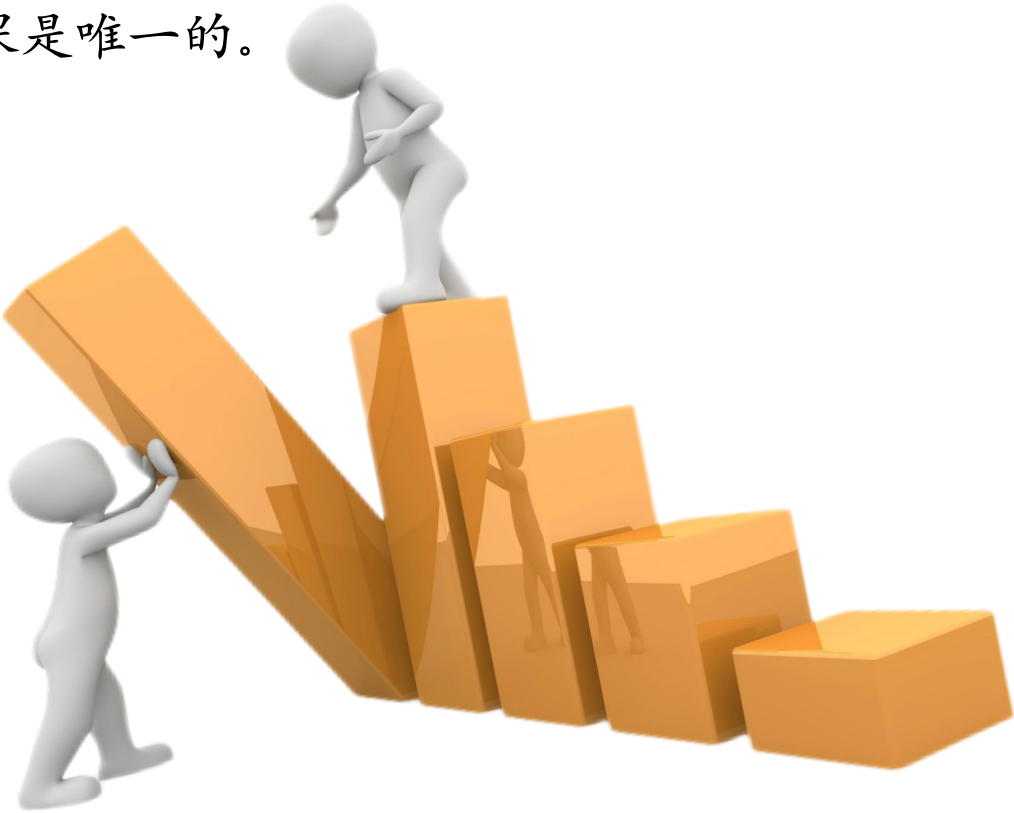
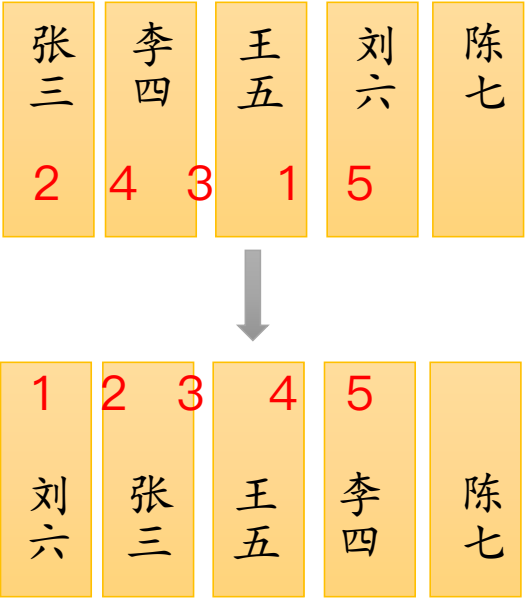
决策树是一棵有 $n!$ 个叶结点的二叉树。

1.1 排序的基本概念

1.1.3 内排序

内排序算法的稳定性

- 1 当待排序元素的关键字均不相同时，排序的结果是唯一的。



1.1 排序的基本概念

1.1.3 内排序

- 2 ➤ 如果待排序的表中，存在有多个关键字相同的元素，经过排序后这些具有相同关键字的元素之间的**相对次序保持不变**，则称这种排序方法是**稳定的**。

3 4 3 1 5



1 2 3 3 5

- 反之，若具有相同关键字的元素之间的**相对次序发生变化**，则称这种排序方法是**不稳定的**。

3 4 3 1 5



1 2 3 3 5

1.1 排序的基本概念

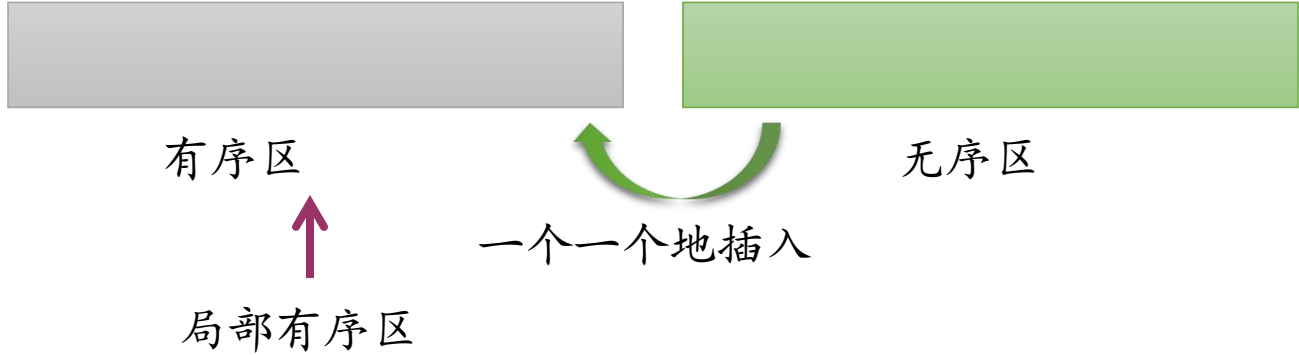
1.1.3 内排序

正序和反序

- 若待排序的表中元素已按关键字排好序，称此表中元素为**正序**；
- 若待排序的表中元素的关键字顺序正好和排好序的顺序相反，称此表中元素为**反序**。



1.2.1 基本思路



有序区类型

- **局部有序区**：其中元素在后面排序中会发生位置的改变。
如 $[3, 6, 8], 1, 5, 7$ ，其中 $[3, 6, 8]$ 为局部有序区
- **全局有序区**：其中元素在后面排序中不再发生位置的改变。
如 $[2, 4, 5], 6, 8, 9$ ，其中 $[2, 4, 5]$ 为全局有序区

1.2.2 主要的插入排序方法

(1) 直接插入排序

(2) 折半插入排序

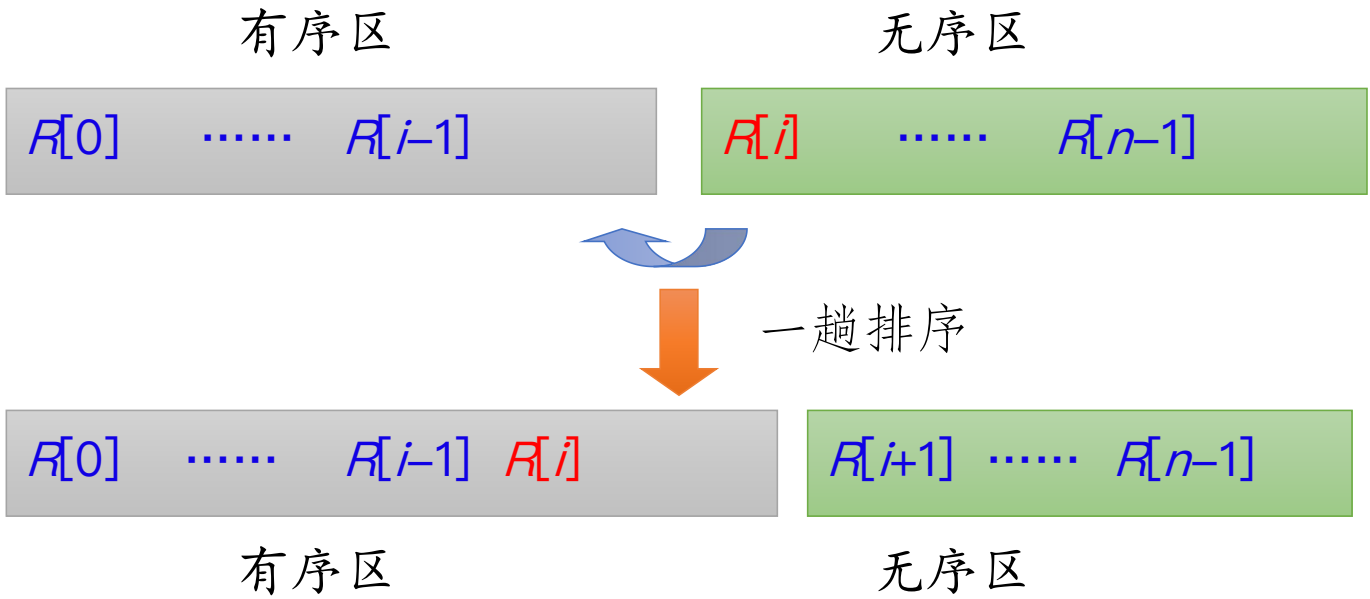
(3) 希尔排序



1.2.2 主要的插入排序方法

1.直接插入排序

1 直接插入排序的基本思路

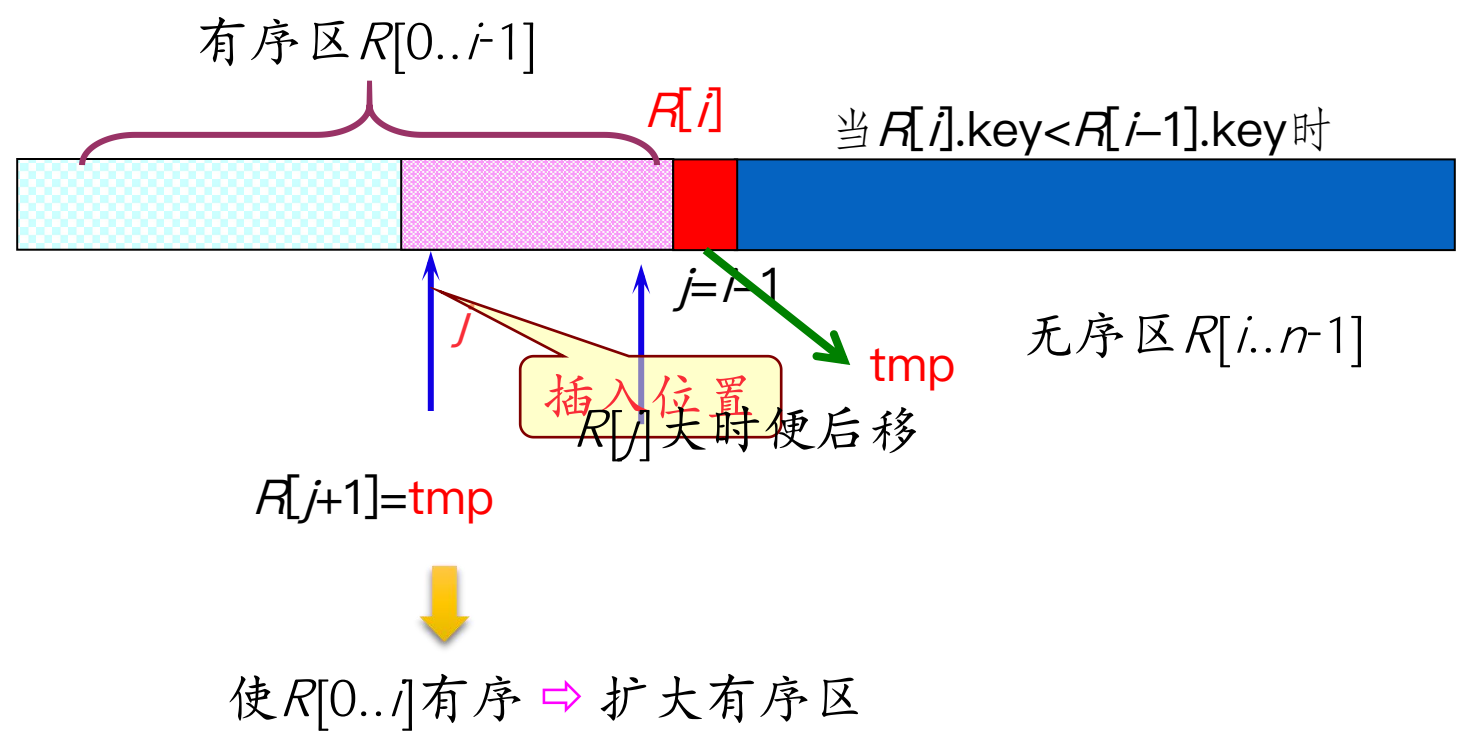


- 初始时，有序区只有一个元素 $R[0]$
- $i = 1 \sim n-1$ ，共经过 $n-1$ 趟排序

1.2.2 主要的插入排序方法

1. 直接插入排序

2 一趟直接插入排序：在有序区中插入 $R[i]$ 的过程

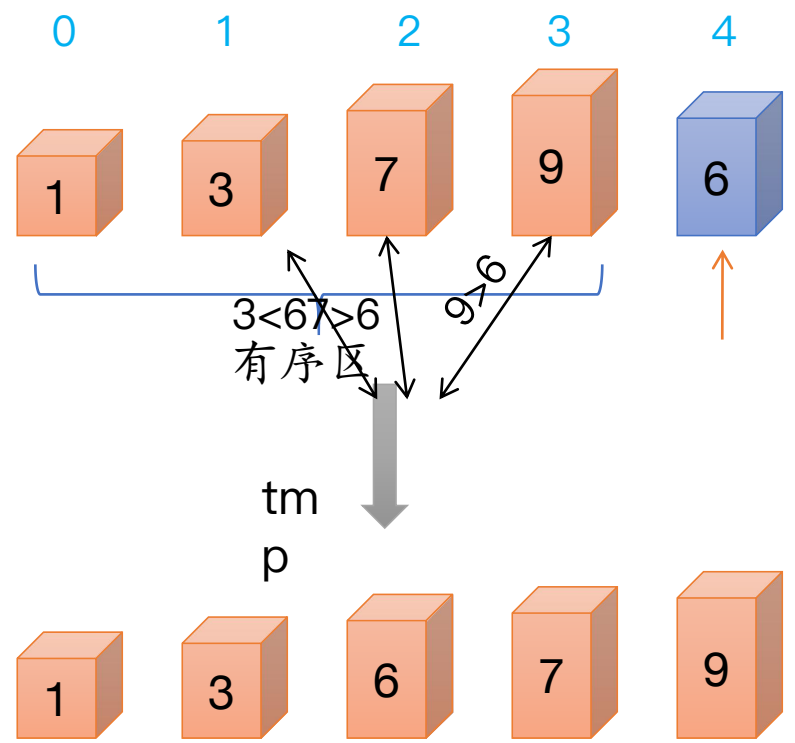


1.2 插入排序

1. 直接插入排序

3 在无序区从后向前查找

例如



😊 在无序区从后向前查找 $\leq \text{tmp.key}$ 的第一个 $R[j]$

1.2.2 主要的插入排序方法

1.. 直接插入排序

4 直接插入排序过程

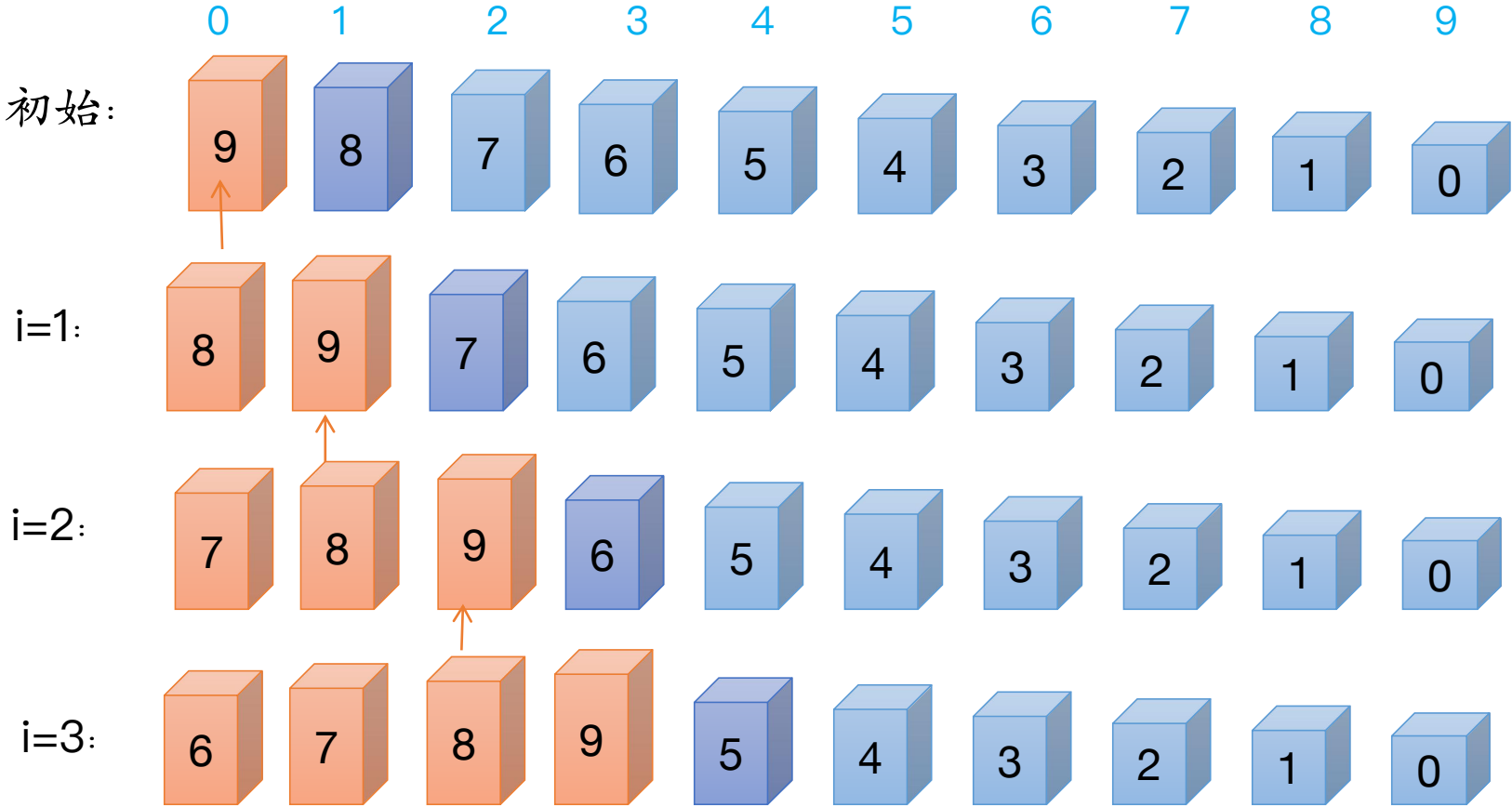
设待排序的表有10个元素，其关键字分别为
(9, 8, 7, 6, 5, 4, 3, 2, 1, 0)。说明采用**直接插入排序**方法进行排序的过程。



1.2.2 主要的插入排序方法

1.直接插入排序

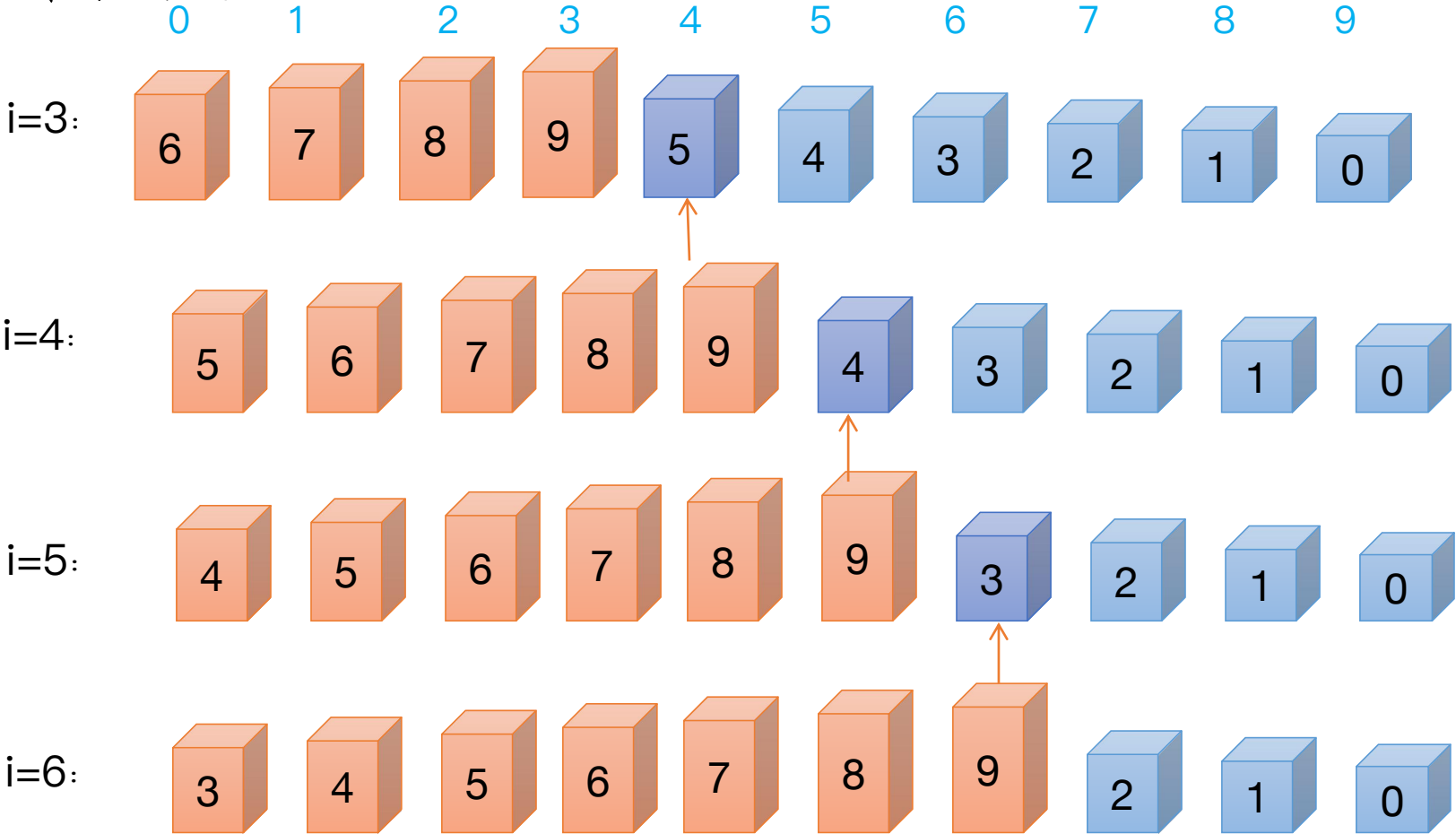
4 直接插入排序的过程



1.2.2 主要的插入排序方法

1. 直接插入排序

4 直接插入排序的过程

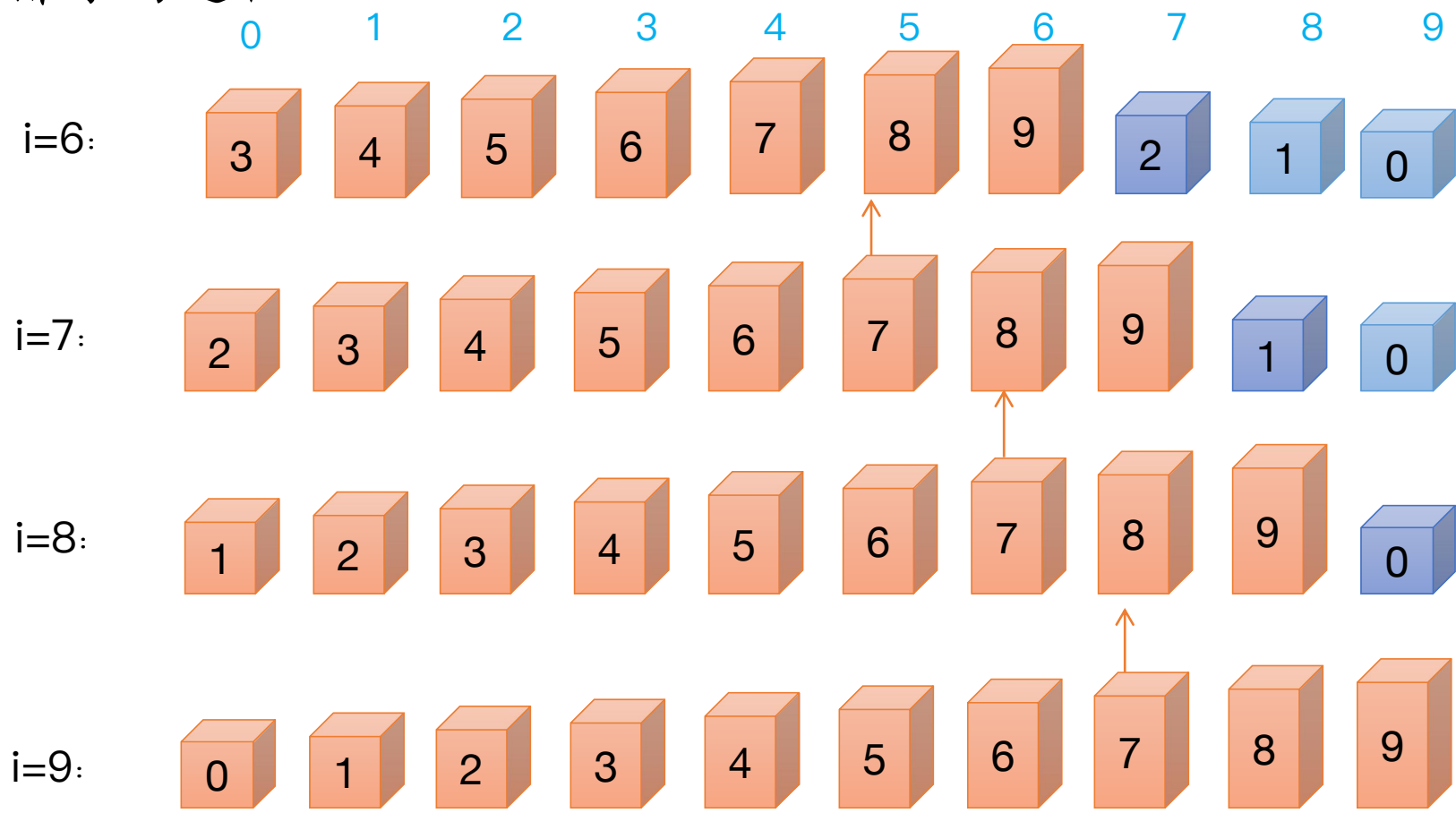


1.2 插入排序

1.2.2 主要的插入排序方法

1. 直接插入排序

4 直接插入排序的过程



已知数据序列{10, 8, 18, 15, 7}, 应用直接插入算法进行排序（正序排序），完成填空。

(1) 每一趟排序的结果如下。在方格中填出相应的数字，实现直接插入算法排序。

10 8 18 15 7

第一趟结束时结果: [8 10] 18 15 7

第二趟结束时结果: [8 10 18] 15 7

第三趟结束时结果: [8 10 15 18] 7

第四趟结束时结果: [7 8 10 15] 18

1.2 插入排序

1.2.2 主要的插入排序方法

1. 直接插入排序

5 直接插入排序的算法

```
void InsertSort(RecType R[], int n)
{
    int i, j;
    RecType tmp;

    for (i=1; i<n; i++) //从第 2 个元素开始，依次将 R[i] 插入到前面已经有序的序列中
    {
        if (R[i].key<R[i-1].key) //反序时
        {
            tmp=R[i]; //将当前待插入元素 R[i] 暂存到 tmp 中，防止后移元素时被覆盖

            j=i-1; //j 指向当前元素前面的那个位置，也就是有序区的最后一个元素

            do //找R[i]的插入位置
            {
                R[j+1]=R[j]; //将关键字大于R[i].key的元素后移，为tmp 腾出插入空间
                j--; // j 向前移动，继续和前面的元素比较
            } while (j>=0 && R[j].key>tmp.key) //只要 j 没有越界，并且 R[j] 仍然大于 tmp，就继续向后移动元素

            R[j+1]=tmp; //在j+1处插入R[i]
        }
    }
}
```

1.2.2 主要的插入排序方法

1.直接插入排序

6

算法分析

最好的情况（关键字在元素序列中正序）：

如 (1,2,3,4,5)

“比较” 的次数：

$$\sum_{i=1}^{n-1} 1 = n - 1$$

“移动” 的次数：

0

最好: $O(n)$



1.2 插入排序

■ 最坏的情况（关键字在元素序列中反序）：

如 (5,4,3,2,1) $R[i]$ 插入到 $R[0..i-1]$ 最前面，比较 i 次，移动 $i+2$ 次

6

算法分析

```
void InsertSort(RecType R[], int n)
{ int i, j; RecType tmp;
  for (i=1; i<n; i++)
  { if (R[i].key < R[i-1].key)      //反序时
    { tmp=R[i];
      j=i-1;
      do                          //找R[i]的插入位置
      { R[j+1]=R[j];              //将关键字大于R[i].key的元素后移
        j--;
      } while (j>=0 && R[j].key>tmp.key)
      R[j+1]=tmp;                 //在j+1处插入R[i]
    }
  }
}
```

“比较” 的次数：

$$\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$$

“移动” 的次数：

$$\sum_{i=1}^{n-1} (i+2) = \frac{(n-1)(n+4)}{2}$$

最坏： $O(n^2)$

■ 平均情况：比较次数+移动次数



6

算法分析

如 $R[0..i-1]$ 中插入 $R[i]$ ，平均插入在中间位置 $R[i/2]$

$$\sum_{i=1}^{n-1} \left(\frac{i}{2} + \frac{i}{2} + 2 \right) = \sum_{i=1}^{n-1} (i + 2) = \frac{(n-1)(n+4)}{2} = O(n^2) \quad \text{平均: } O(n^2)$$

1.2 插入排序

1.2.2 主要的插入排序

2..折半插入排序

1 折半插入排序基本思路

查找采用折半查找方法，称为二分插入排序或折半插入排序。



1.2.2 主要的插入排序方法

2..折半插入排序 ② 如何在 $R[\text{low}..\text{high}]$ 中查找插入 $R[i]$ 的位置?

在 $R[\text{low}..\text{high}]$ 中查找 $\geq R[i].\text{key}$ ($k=R[i].\text{key}$) 的 **最后的位置** high

① 如果 $R[\text{low}..\text{high}]$ 不空:

$$\text{mid}=(\text{low}+\text{high})/2$$

$$k < R[\text{mid}].\text{key}$$

$$R[\text{low}..\text{mid}-1] < k$$

$$k \geq R[\text{mid}].\text{key}$$

$$R[\text{mid}+1..\text{high}] \geq k$$

② 如果 $R[\text{low}..\text{high}]$ 空:

$$R[\text{low}..\text{high}] \quad R[\text{high}+1..*]$$

↑
 $R[i]$



1.2.2 主要的插入排序方法

2. 折半插入排序

3 折半插入排序算法



```
void BinInsertSort(RecType R[], int n)
{ int i, j, low, high, mid;
  RecType tmp;
  for (i=1; i<n; i++)
  { if (R[i].key<R[i-1].key) //反序时
    { tmp=R[i]; //将R[i]保存到tmp中
      low=0; high=i-1;
      while (low<=high) //在R[low..high]中查找插入的位置
      { mid=(low+high)/2; //取中间位置
        if (tmp.key<R[mid].key)
          high=mid-1; //插入点在左半区
        else
          low=mid+1; //插入点在右半区
      } //找位置high+1
      for (j=i-1; j>=high+1; j--) //元素后移
        R[j+1]=R[j];
      R[high+1]=tmp; //插入tmp
    }
  }
}
```

1.2.2 主要的插入排序方法

2..折半插入排序

- 在 $R[0..i-1]$ 中查找插入 $R[i]$ 的位置，折半查找的平均关键字比较次数为 $\log_2(i+1)-1$ 。
- 平均移动元素的次数为 $i/2+2$ 。



$$\sum_{i=1}^{n-1} [\log_2(i+1) - 1 + \frac{i}{2} + 2] = O(n^2)$$

- 折半插入排序采用折半查找，查找效率提高，即关键字比较次数减少了。
- 但元素移动次数不变，仅仅将分散移动改为集合移动。

1.2.2 主要的插入排序方法

2..折半插入排序

示例

对同一待排序序列分别进行折半插入排序和直接插入排序，两者之间可能的不同之处是（ ）。

- A.排序的总趟数
- B.元素的移动次数
- C.使用辅助空间的数量
- D.元素之间的比较次数

答案D。说明：本题为2012年全国考研题



已知数据序列{10, 8, 18, 15, 7}, 采用折半插入算法进行排序, 每一趟排序结束后的结果是什么?

第一趟: $8 < 10$ 排序8 排序结果: 8 10 18 15 7

第二趟: $18 > 10$ 排序18 排序结果: 8 10 18 15 7

第三趟: $10 < 15 < 18$ 排序15 排序结果: 8 10 15 18 7

第四趟: $7 < 10; 7 < 8$ 排序7 排序结果: 7 8 10 15 18

1.2.2 主要的插入排序方法

3..希尔排序

1 希尔排序的基本思路

- ① $d=n/2$
- ② 将排序序列分为 d 个组，在各组内进行直接插入排序
- ③ 递减 $d=d/2$ ，重复②，直到 $d=1$



算法最后一趟对所有数据进行了直接插入排序，所以结果一定是正确的。



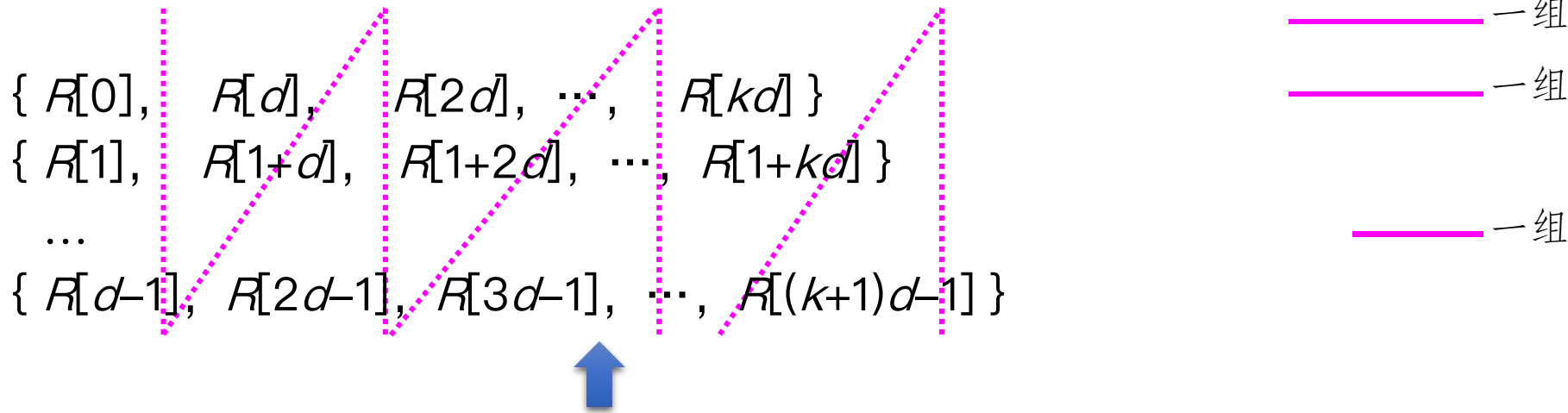
1.2.2 主要的插入排序方法

3..希尔排序

2 希尔排序的过程

将元素序列分成若干子序列， 分别对每个子序列进行直接插入排序。

例如：将 n 个元素分成 d 个子序列：



1.2 插入排序

1.2.2 主要的插入排序方法

3..希尔排序

初始序列	9	8	7	6	5	4	3	2	1	0
$d=5$	9	8	7	6	5	4	3	2	1	0
直接插入排序	4	3	2	1	0	9	8	7	6	5
$d=d/2=2$	4	3	2	1	0	9	8	7	6	5
直接插入排序	0	1	2	3	4	5	6	7	8	9
$d=d/2=1$	0	1	2	3	4	5	6	7	8	9
直接插入排序	0	1	2	3	4	5	6	7	8	9

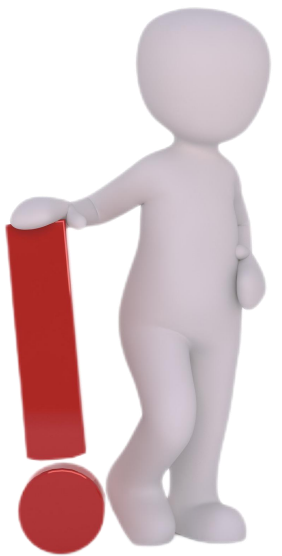
2 希尔排序的过程

注意：对于 $d=1$ 的一趟，排序前的数据已将近正序！

1.2.2 主要的插入排序方法

3..希尔排序

3 希尔排序算法



```
void ShellSort(RecType R[], int n)
{ int i, j, d;
  RecType tmp;
  d=n/2;                                //增量置初值
  while (d>0)
  { for (i=d;i<n;i++)
    { //对相隔d位置的元素组直接插入排序
      tmp=R[i];
      j=i-d;
      while (j>=0 && tmp.key<R[j].key)
      { R[j+d]=R[j];
        j=j-d;
      }
      R[j+d]=tmp;
    }
    d=d/2;                              //减小增量
  }
}
```

d循环: 每个元素都参加排序了

A blue curved arrow starts from the text "d循环: 每个元素都参加排序了" and points towards the inner loop of the ShellSort code, specifically the part where elements are shifted and inserted.

```
for (i=1;i<n;i++)
{ tmp=R[i];
  j=i-1;
  while (j>=0 && tmp.key<R[j].key)
  { R[j+1]=R[j];
    j=j-1;
  }
  R[j+1]=tmp;
}
```

1.2.2 主要的插入排序方法

3. 希尔排序

4 为什么希尔排序比直接排序好

希尔排序的时间复杂度约为 $O(n^{1.3})$ 。

为什么希尔排序比直接插入排序好？

例如：有10个元素要排序。



希尔排序

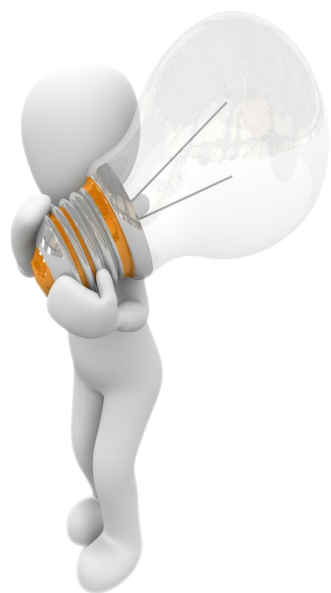
直接插入排序

大约时间 $= 10^2 = 100$

$d=5$: 分为5组, 时间约为 $5 \times 2^2 = 20$

$d=2$: 分为2组, 时间约为 $2 \times 5^2 = 50$

$d=1$: 分为1组, 几乎有序, 时间约为 10



1.2.2 主要的插入排序方法

3..希尔排序

4 为什么希尔排序比直接排序好

■ n 个元素直接插入排序的平均时间复杂度为 $O(n^2)$

- 分为 d 组，每组 n/d 个元素：时间为 $d(n/d)^2 = n^2/d$
- 分组后的时间和远小于 n 个元素直接插入排序时间，并且 d 越大时间和越少

■ d 越小时越接近有序，当 $d=1$ 时基本有序。

直接插入排序在越有序时时间越少，趋向 $O(n)$



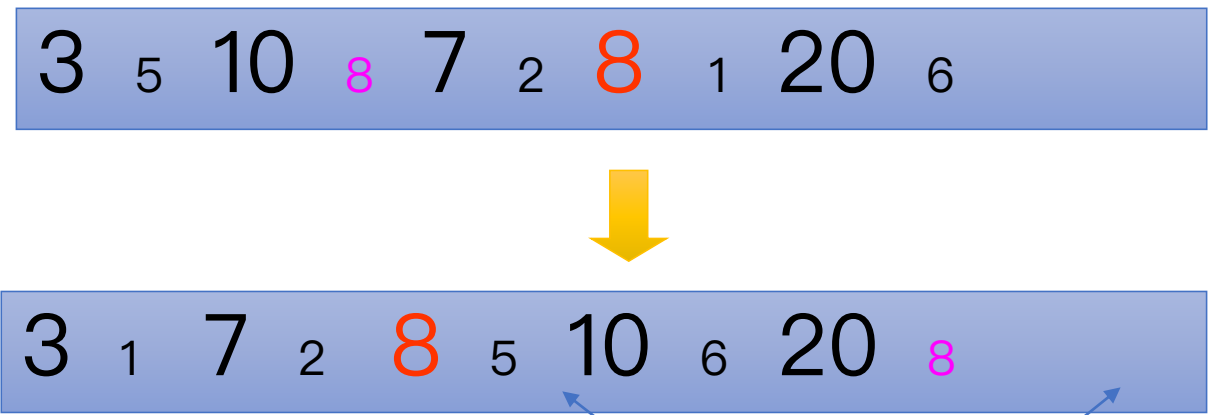
1.2 插入排序

1.2.2 主要的插入排序方法

5. 希尔排序算法不稳定

希尔排序算法不稳定： 希尔排序法是一种不稳定的排序算法。

$d=2$



相对位置发生改变



希尔排序是不稳定的!

1.2.2 主要的插入排序方法

3..希尔排序

5 希尔排序算法案例



希尔排序的组内排序采用的是 ()

- A.直接插入排序
- B.折半插入排序
- C.快速排序
- D.归并排序

答案：A 说明：本题为2015年全国考研题



数据结构

教师: 刘芳岑

邮箱: liufc@hainanu.edu.cn

学院: 计算机科学与技术学院

时间: 2026春季学期



课堂回顾



➤ 插入排序

(1) 直接插入排序

(2) 折半插入排序

(3) 希尔排序



有一组关键码序列 (15,8,19,16,25,31,7,23,18,20) ，采用直接插入排序、希尔排序方法由小到大进行排序，请写出每趟的结果。

第1趟 8, 15, 19, 16, 25, 31, 7, 23, 18, 20

第2趟 8, 15, 19, 16, 25, 31, 7, 23, 18, 20

第3趟 8, 15, 16, 19, 25, 31, 7, 23, 18, 20

第4趟 8, 15, 16, 19, 25, 31, 7, 23, 18, 20

第5趟 8, 15, 16, 19, 25, 31, 7, 23, 18, 20

第6趟 7, 8, 15, 16, 19, 25, 31, 23, 18, 20

第7趟 7, 8, 15, 16, 19, 23, 25, 31, 18, 20

第8趟 7, 8, 15, 16, 18, 19, 23, 25, 31, 20

第9趟 7, 8, 15, 16, 18, 19, 20, 23, 25, 31

	增量 d	排序结果
第1趟	5	15, 7, 19, 16, 20, 31, 8, 23, 18, 25
第2趟	2	8, 7, 15, 16, 18, 23, 19, 25, 20, 31
第3趟	1	7, 8, 15, 16, 18, 19, 20, 23, 25, 31



课堂预告



➤ 交换排序

(1) 冒泡排序

(2) 快速排序

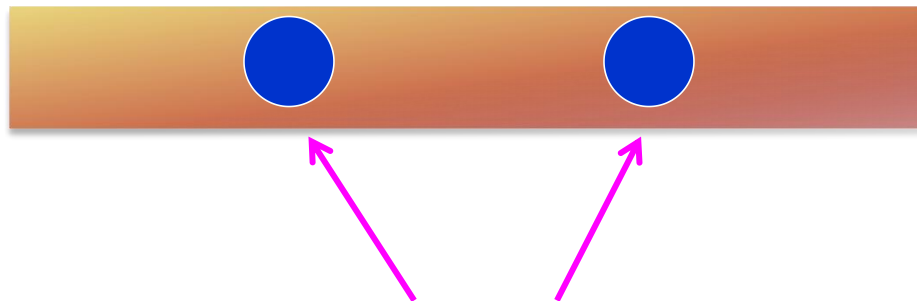
➤ 选择排序

(1) 简单选择排序

(2) 堆排序

10.3 交换排序

基本思路



两个元素反序时进行交换

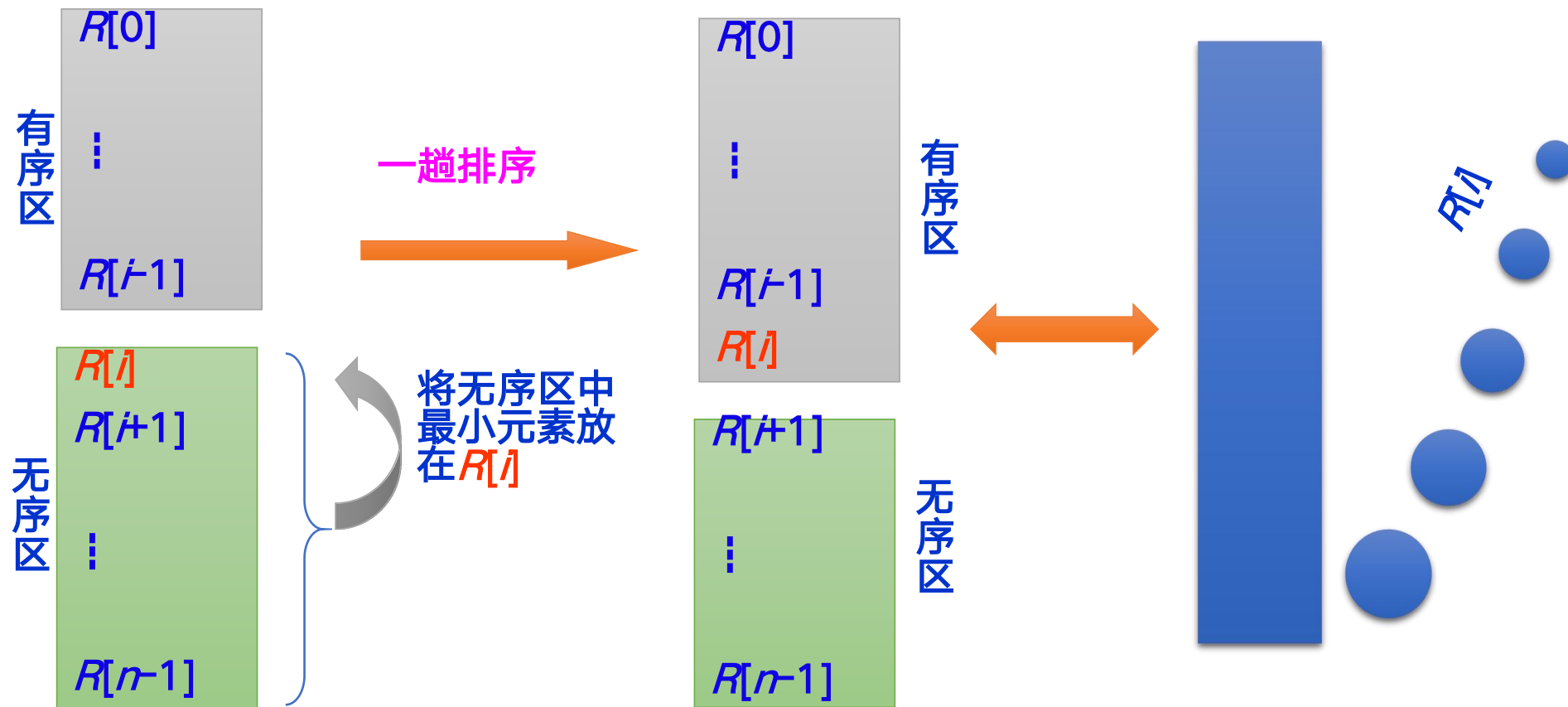
常见的交换排序方法：

- (1) 冒泡排序（或起泡排序）
- (2) 快速排序



10.3 交换排序

10.3.1 冒泡排序



- 初始有序区为空。
- $i = 0 \sim n-2$, 共 $n-1$ 趟使整个数据有序。

有序区总是全局有序的

10.3 交换排序

10.3.1 冒泡排序-算法



```
void BubbleSort(RecType R[], int n)
{ int i, j; RecType temp;
  for (i=0;i<n-1;i++)
  {
    for (j=n-1;j>i;j--)           //比较找本趟最小关键字的元素
      if (R[j].key<R[j-1].key)    //反序
        swap(R[j],R[j-1]);       //R[j]↔R[j-1]
  }
}
```

10.3 交换排序

10.3.1 冒泡排序-算法

采用前面的冒泡排序方法对(2, 1, 3, 4, 5) 进行排序

初始关键字



$i=0$



$i=1$



$i=2$



$i=3$



已经全部
有序了



如何提高效率？

一旦某一趟比较时不出现元素交换，说明已排好序了，就可以结束本算法。

10.3 交换排序

10.3.1 冒泡排序-算法

```
void BubbleSort(RecType R[], int n)
{ int i, j; bool exchange; RecType temp;
  for (i=0;i<n-1;i++)
  {
    exchange=false;
    for (j=n-1;j>i;j--)           //比较, 找出最小关键字的元素
      if (R[j].key<R[j-1].key)    //反序⇒交换
      { swap(R[j],R[j-1]);
        exchange=true;
      }
    if (exchange==false) return; //中途结束算法
  }
}
```



10.3 交换排序

10.3.1 冒泡排序-算法分析

- 最好的情况（关键字在元素序列中正序）：只需进行一趟冒泡

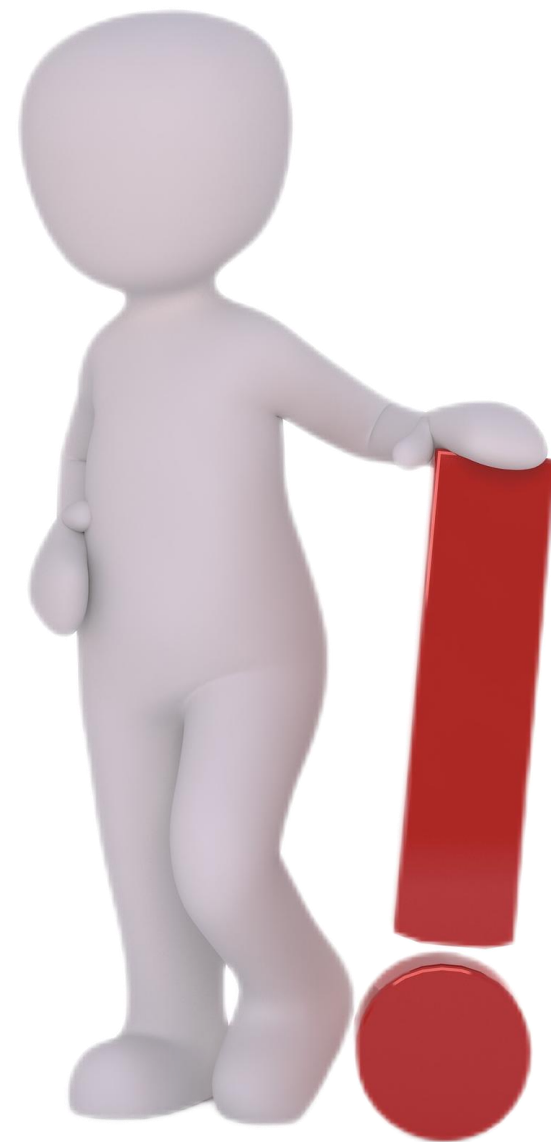
“比较”的次数：

$n-1$

时间复杂度为 $O(n)$

“移动”的次数：

0



10.3 交换排序

10.3.1 冒泡排序-算法分析

■ 最坏的情况（关键字在元素序列中反序）：需进行 $n-1$ 趟冒泡

```
void BubbleSort(RecType R[], int n)
{ int i, j; bool exchange; RecType temp;
  for (i=0;i<n-1;i++)
  { exchange=false;
    for (j=n-1;j>i;j--) //R[i..n-1]:n-i个元素,比较次数n-i-1
      if (R[j].key<R[j-1].key) //反序⇒交换
      { swap(R[j],R[j-1]); //交换: 3次移动
        exchange=true;
      }
    if (exchange==false) return; //中途结束算法
  }
}
```

“比较”的次数：

$$\sum_{i=0}^{n-2} (n-i-1) = \frac{n(n-1)}{2}$$

“移动”的次数：

$$\sum_{i=0}^{n-2} 3(n-i-1) = \frac{3n(n-1)}{2}$$

时间复杂度为 $O(n^2)$

10.3 交换排序

10.3.1 冒泡排序-算法分析

平均情况

- 算法可能在中间的某一道排序完成后就终止，但可以证明平均的排序趟数仍是 $O(n)$ ，
- 由此得出平均情况下，总的比较次数仍是 $O(n^2)$ ，故算法的平均时间复杂度为 $O(n^2)$ 。



所以冒泡排序最好时间复杂度为 $O(n)$ ，最坏和平均为 $O(n^2)$ 。

算法分析

- 设有一组初始关键字序列为(16, 10, 8, 9, 21, 0, 5), 则第3趟冒泡排序(按递增排序, 从左往右冒泡)结束后的结果是

_____。

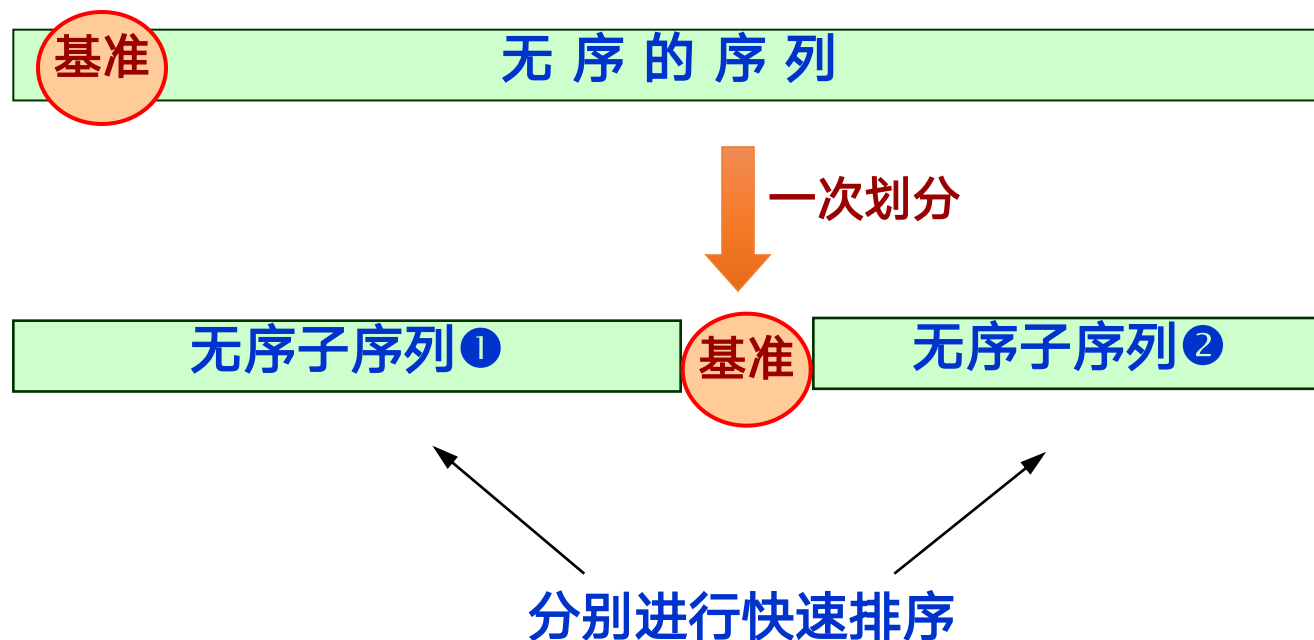
8, 9, 0, 5, 10, 16, 21

有一组关键字19、16、25、37、78、32、41、10，采用冒泡排序，从小到大排序，从右往左冒泡。写出每一趟的序列结果

第1趟	10, 19, 16, 25, 37, 78, 32, 41
第2趟	10, 16, 19, 25, 32, 37, 78, 41
第3趟	10, 16, 19, 25, 32, 37, 41, 78
第4趟	10, 16, 19, 25, 32, 37, 41, 78
第5趟	10, 16, 19, 25, 32, 37, 41, 78
第6趟	10, 16, 19, 25, 32, 37, 41, 78
第7趟	10, 16, 19, 25, 32, 37, 41, 78

10.3 交换排序

10.3.2 快速排序



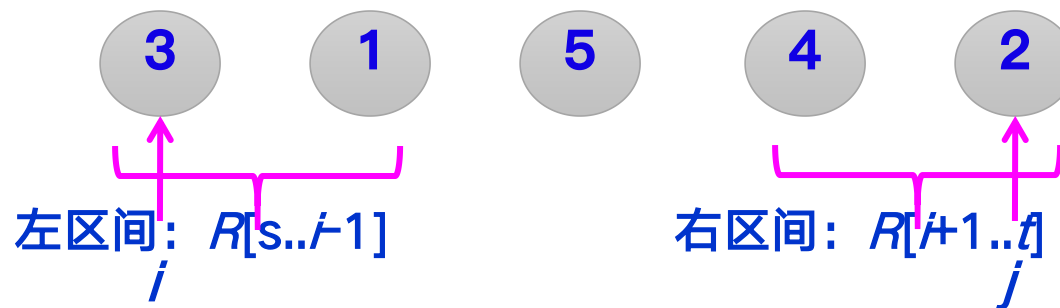
- 每趟使表的**第1个元素（基准）**放入适当位置（归位），将表一分为二，对子表按递归方式继续这种划分。
- 直至划分的子表长为0或1（递归出口）。

10.3 交换排序

10.3.2 快速排序

回顾划分：示例

整个区间： $R[s..t]$



$i=j$: 区间处理完毕

划分完毕

10.3 交换排序

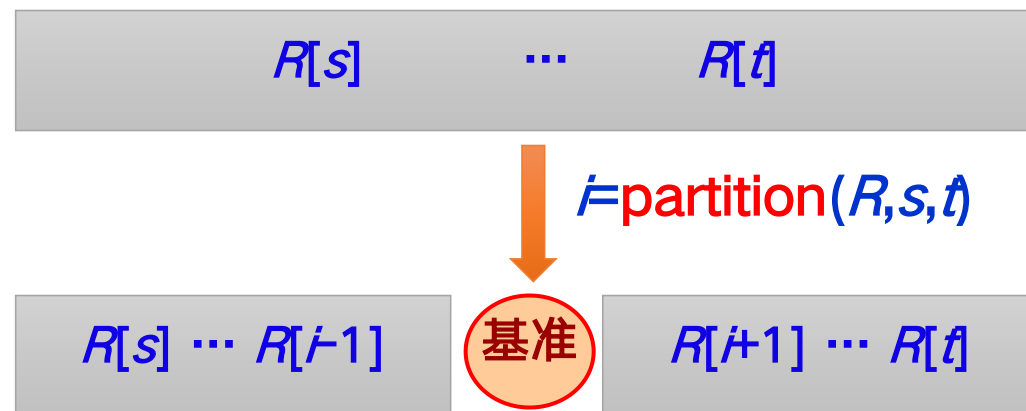
10.3.2 快速排序-算法

```
int partition(RecType R[],int s,int t) //一趟划分
{
    int i=s,j=t;
    RecType tmp=R[i]; //以R[i]为基准
    while (i<j) //从两端交替向中间扫描,直至i=j为止
    {
        while (j>i && R[j].key>=tmp.key)
            j--; //从右向左扫描,找一个小于tmp.key的R[j]
        if(i<j)
        {
            R[i]=R[j]; //找到这样的R[j],放入R[i]处
            j--;
        }
        while (i<j && R[i].key<=tmp.key)
            i++; //从左向右扫描,找一个大于tmp.key的R[i]
        if(i<j)
        {
            R[j]=R[i]; //找到这样的R[i],放入R[j]处
            i++;
        }
    }
    R[i]=tmp;
    return i;
}
```



10.3 交换排序

10.3.2 快速排序-算法

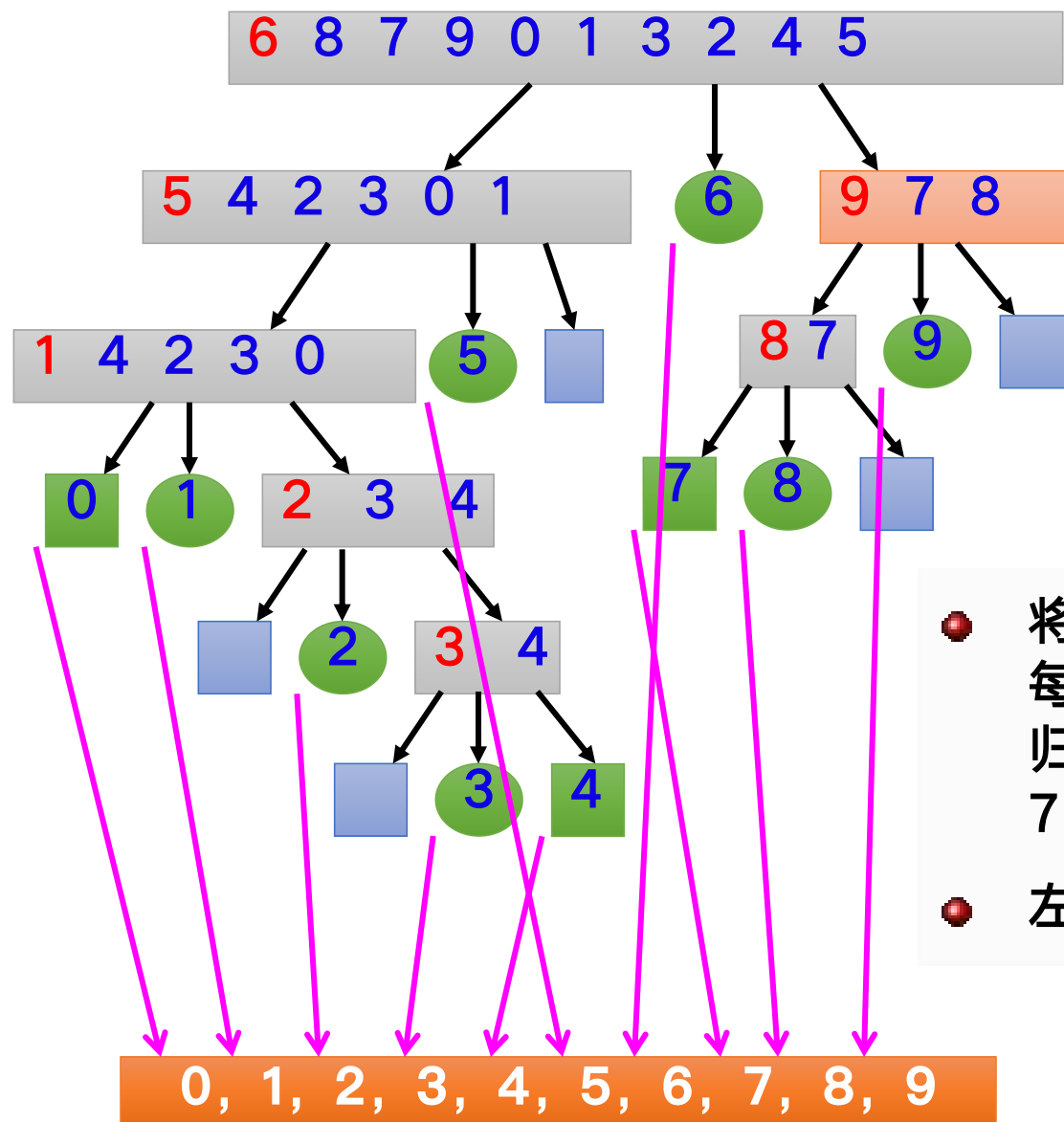


```
void QuickSort(RecType R[],int s,int t)
//对R[s..t]的元素进行快速排序
{ int i;
  if (s<t) //区间内至少存在两个元素的情况
  { i=partition(R,s,t);
    QuickSort(R,s,i-1); //对左区间递归排序
    QuickSort(R,i+1,t); //对右区间递归排序
  }
}
```

【例10-4】 设待排序的表有10个记录，其关键字分别为 (6, 8, 7, 9, 0, 1, 3, 2, 4, 5)。说明采用快速排序方法进行排序的过程。

10.3 交换排序

10.3.2 快速排序-递归树



- 将递归树看成一颗3叉树，每个分支结点对应一次递归调用。这里递归次数：7
- 左右分区处理的顺序无关

10.3 交换排序

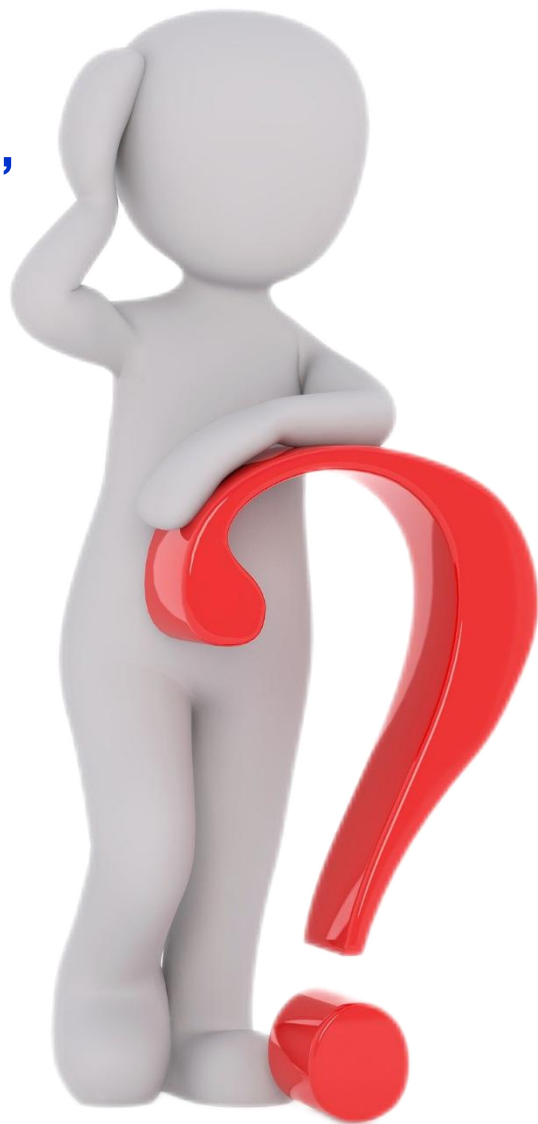
10.3.2 快速排序-递归树

示例

采用递归方式对顺序表进行快速排序，下列关于递归次数的叙述中，正确的是（ ）。

- A. 递归次数与初始数据的排列次序无关
- B. 每次划分后，先处理较长的分区可以减少递归次数
- C. 每次划分后，先处理较短的分区可以减少递归次数
- D. 递归次数与每次划分后得到的分区处理顺序无关

D 说明：本题为2010年全国考研题



10.3 交换排序

10.3.2 快速排序-递归树

示例

为实现快速排序法，待排序序列宜采用存储方式是（ ）。

A. 顺序存储

B. 散列存储

C. 链式存储

D. 索引存储

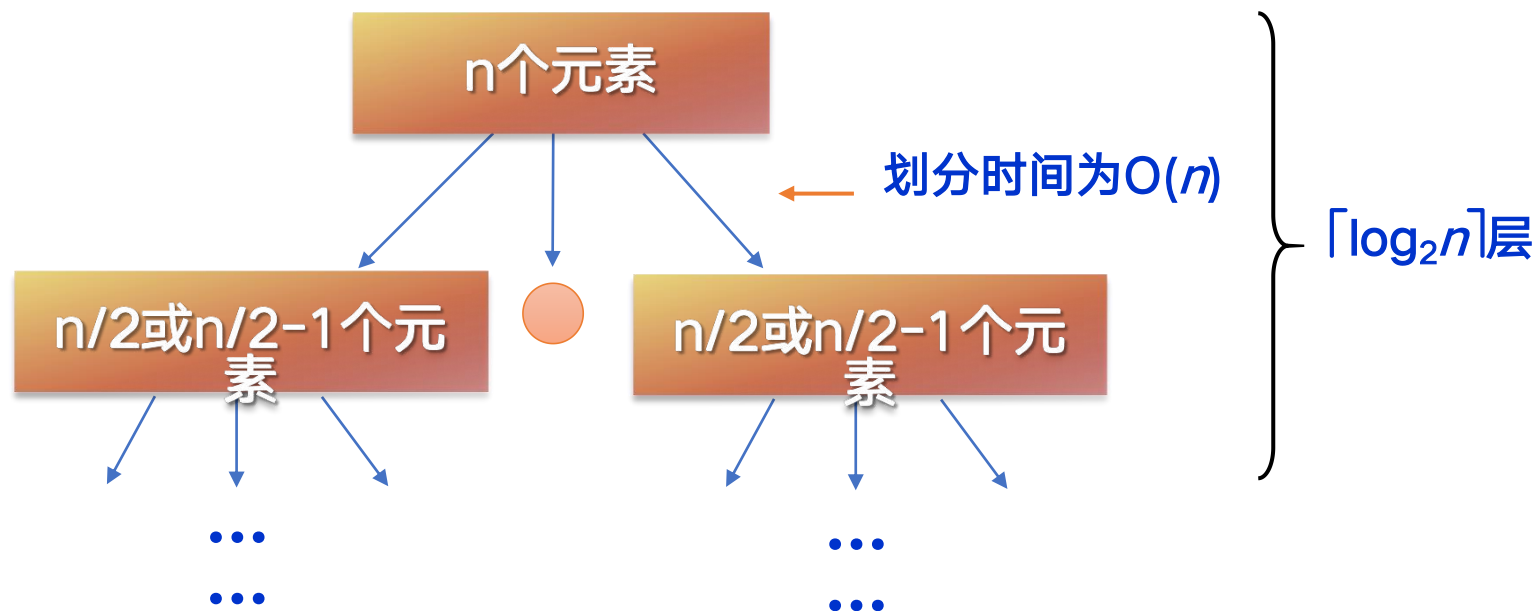
A 说明：本题为2011年全国考研题



10.3 交换排序

10.3.2 快速排序-算法分析

■ 最好情况:



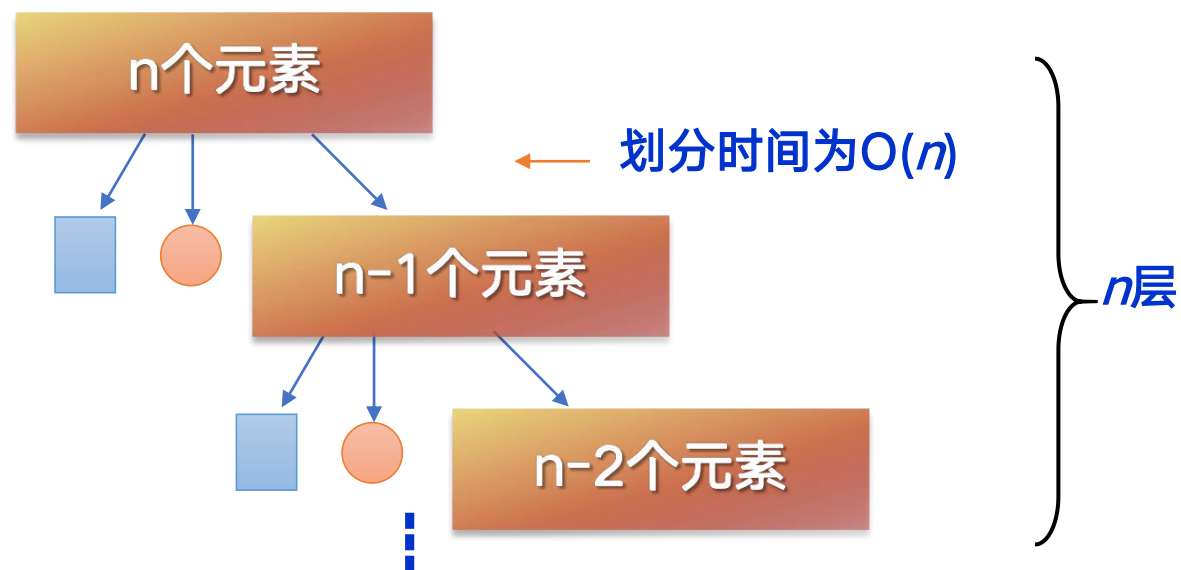
此时时间复杂度为 $O(n \log_2 n)$ ，空间复杂度为 $O(\log_2 n)$ 。

每一层空间复杂度为 $O(1)$ ，一共有 $\log_2 n$ 层。

10.3 交换排序

10.3.2 快速排序-算法分析

■ 最坏情况:

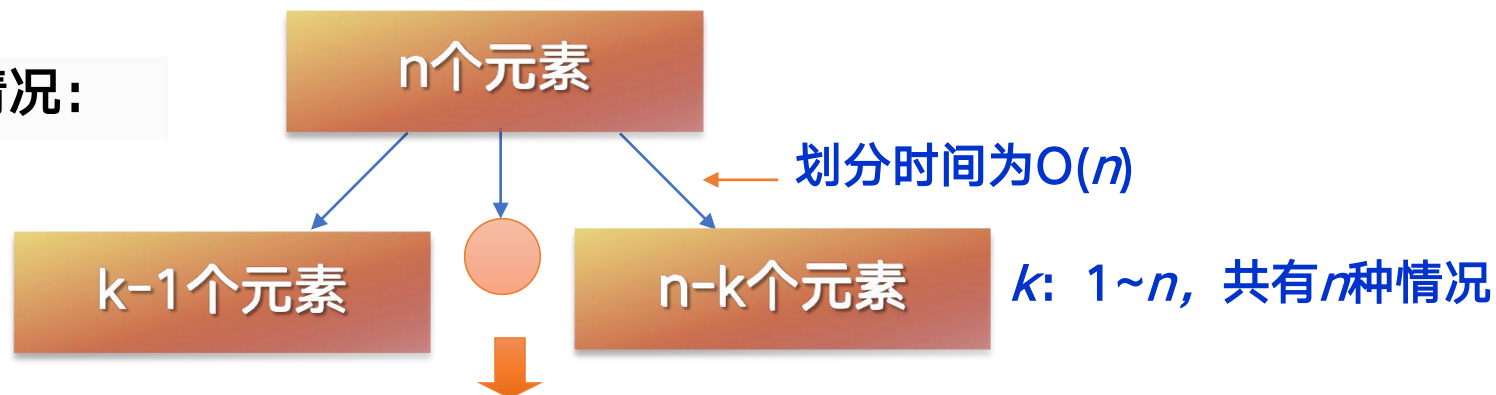


此时时间复杂度为 $O(n^2)$ ，空间复杂度为 $O(n)$ 。

10.3 交换排序

10.3.2 快速排序-算法分析

■ 平均情况:



由此可得快速排序所需时间的平均值为:

$$T_{avg}(n) = \underset{\substack{\uparrow \\ \text{1次划分的时间}}}{Cn} + \frac{1}{n} \sum_{k=1}^n [T_{avg}(k-1) + T_{avg}(n-k)]$$

则可得结果: $T_{avg}(n) = Cn \log_2 n$ 。

结论: 快速排序的平均时间复杂度为 $O(n \log_2 n)$ 。

平均所需栈空间为 $O(\log_2 n)$ 。



下列序列中,()是执行第一趟快速排序后所得的序列 (假设按大到小的顺序进行排序,绿色为基准) .

A.[68,11,18,69] [23,93,73] B.[68,11,69,23] [18,93,73]

C.[93,73] [68,11,69,23,18] D.[68,11,69,23,18] [93,73]

答案: C



📖 3. 设一组初始关键字为 (72,73,71,23,94,16,5) 的序列, 写出快速排序的每一趟结果

第一趟, 基准72: 5, 16, 71, 23, 72, 94, 73
第二趟, 基准5: 5, 16, 71, 23, 72, 94, 73
第三趟, 基准16: 5, 16, 71, 23, 72, 94, 73
第四趟, 基准71: 5, 16, 23, 71, 72, 94, 73
第五趟, 基准94: 5, 16, 23, 71, 72, 73, 94

10.3 交换排序

10.3.2 快速排序-算法分析

示例

2016年全国计算机学科专业考研题

已知由 n ($n \geq 2$) 个正整数构成的集合 $A = \{a_k\}$ ($0 \leq k < n$)，将其划分为两个不相交的子集 A_1 和 A_2 ，元素个数分别是 n_1 和 n_2 ， A_1 和 A_2 中元素之和分别为 S_1 和 S_2 。设计一个尽可能高效的划分算法，满足 $|n_1 - n_2|$ 最小且 $|S_1 - S_2|$ 最大。要求：

(1) 给出算法的基本设计思想。

(2) 根据设计思想，采用C、C++描述算法，关键之处给出注释。

思路：将最小的 $\lfloor n/2 \rfloor$ 个元素放在 A_1 中，其他放在 A_2 中

查找第 $n/2$ 小的元素

递归快速排序

10.3 交换排序

10.3.2 快速排序-算法分析

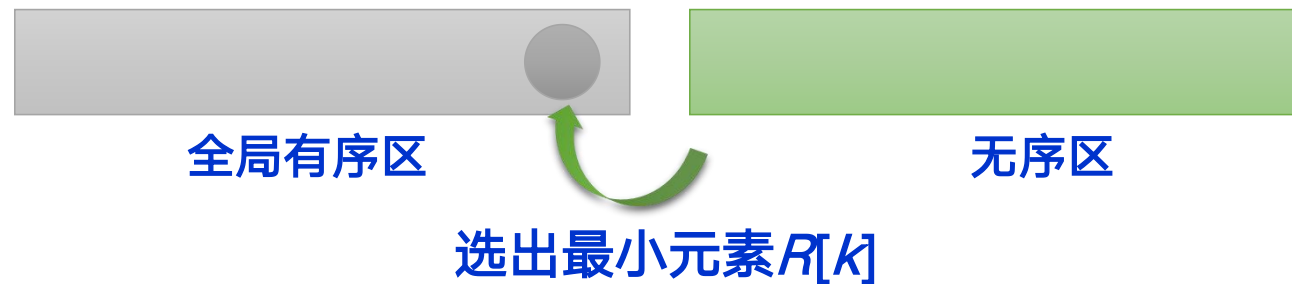
```
int Partition(int a[],int low,int high) //以a[low]为基准划分
{
    int i=low,j=high;
    int pivot=a[low];
    while (i<j)
    {
        while (i<j && a[j]>=pivot)
            j--;
        a[i]=a[j];
        while (i<j && a[i]<=pivot)
            i++;
        a[j]=a[i];
    }
    a[i]=pivot;
    return i;
}
```

10.3 交换排序

10.3.2 快速排序-算法分析

```
int Solution(int a[],int n)           //求解算法
{
    int low=0,high=n-1;
    bool flag=true;
    while (flag)
    {
        int i=Partition(a,low,high); //对当前区间进行一次划分, 返回基准元素最终位置 i
        if (i==n/2-1)                 //基准a[i]为第n/2的元素
            flag=false;               //划分结束
        else if (i<n/2-1)              //在右区间查找
            low=i+1;
        else                           //在左区间查找
            high=i-1;
    }
    int s1=0,s2=0;
    for (int i=0;i<n/2;i++)
        s1+=a[i];
    for (int j=n/2;j<n;j++)
        s2+=a[j];
    return s2-s1;
}
```

10.4 选择排序



常见的选择排序方法：

- (1) 简单选择排序（或称直接选择排序）
- (2) 堆排序

10.4 选择排序

10.4.1 简单选择排序

从 $a[i..n-1]$ 中选出最小元素：

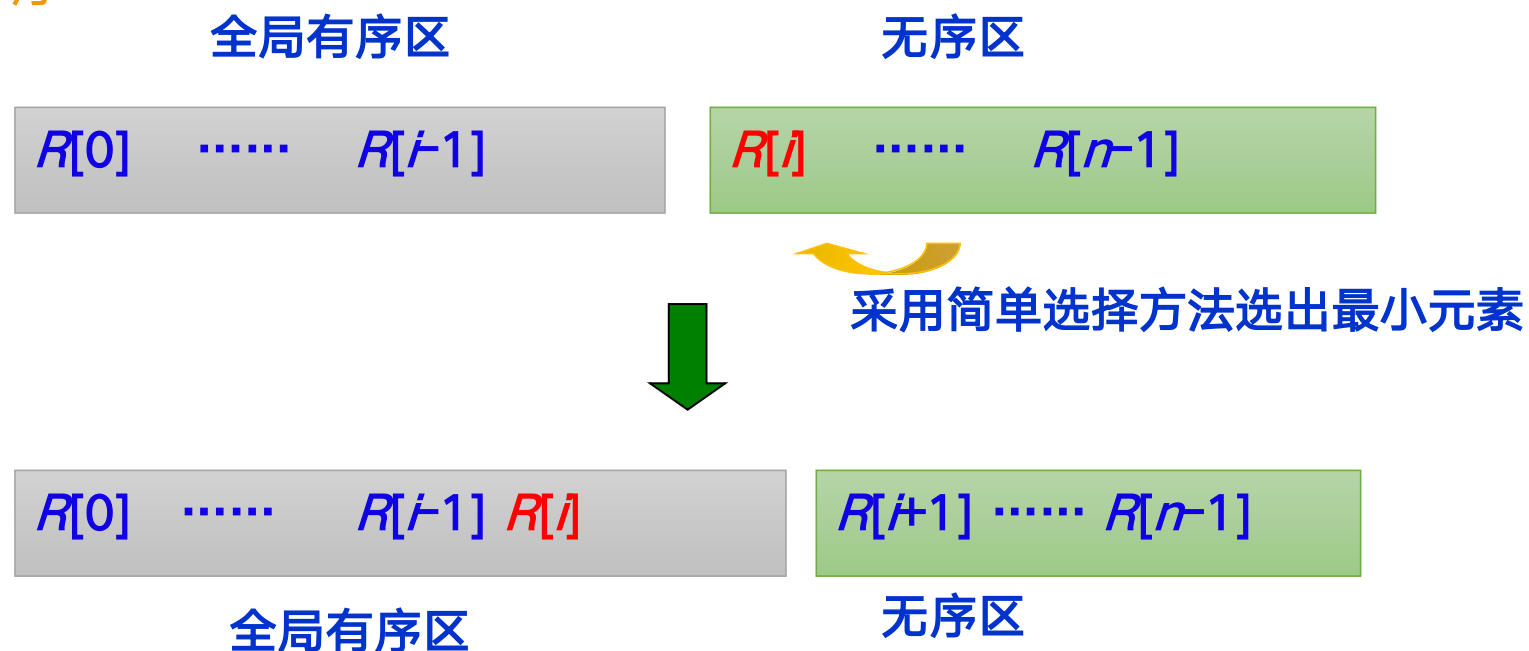
```
int Min(int a[], int n, int i)
{ int k=i, j;           //k保存最小元素的下标
  for (j=i+1;j<n;j++)
    if (a[j]<a[k]) k=j;
  return a[k];
}
```

简单选择

n 个元素中找最小元素需要 $n-1$ 次比较

10.4 选择排序

10.4.1 简单选择排序



- 初始时，全局有序区为空
- $i = 0 \sim n-2$ ，共经过 $n-1$ 趟排序

10.4 选择排序

10.4.1 简单选择排序-算法

```
void SelectSort(RecType R[], int n)
{ int i, j, k; // i 表示当前排序趟数, j 用于遍历, k 用于记录最小元素的位置
  for (i=0;i<n-1;i++)      //共进行 n-1 趟排序
  { k=i; // 假设当前未排序区间的第一个元素 R[i] 最小
    for (j=i+1;j<n;j++) // 在未排序区间 R[i+1] 到 R[n-1] 中查找最小元素
      if (R[j].key<R[k].key) // 如果找到比当前最小元素还小的元素
        k=j; // 更新最小元素的位置
    if (k!=i)      // 如果最小元素的位置不是 i, 说明需要交换
      swap(R[i],R[k]); // 交换
  }
}
```

10.4 选择排序

10.4.1 简单选择排序-算法

采用简单选择排序方法对(2, 1, 3, 4, 5) 进行排序

初始关键字

2	1	3	4	5
---	---	---	---	---

$i=0$

1	2	3	4	5
---	---	---	---	---

$i=1$

1	2	3	4	5
---	---	---	---	---

$i=2$

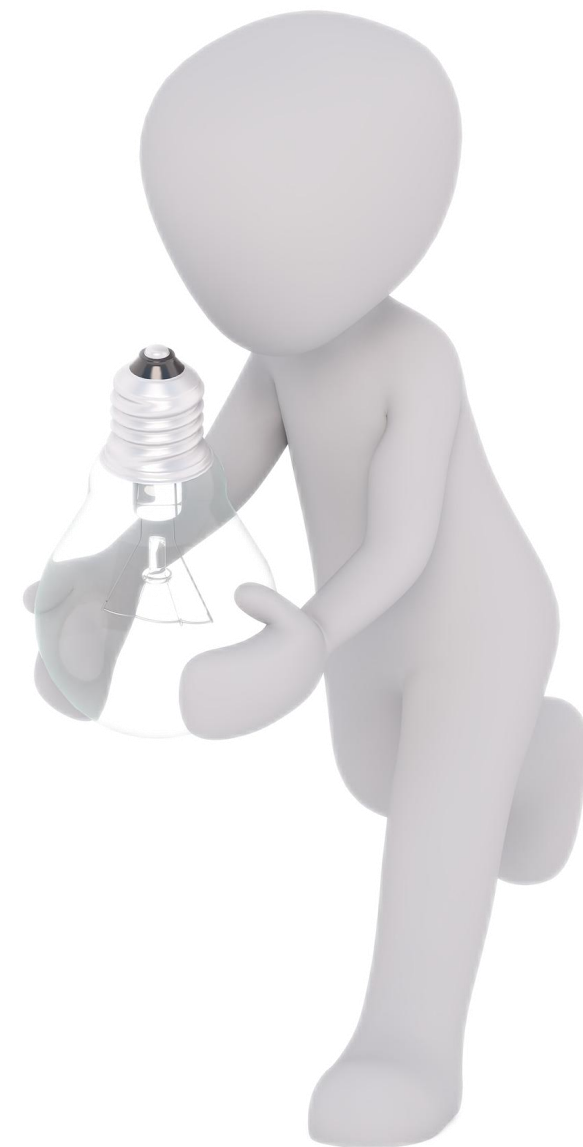
1	2	3	4	5
---	---	---	---	---

$i=3$

1	2	3	4	5
---	---	---	---	---

没有元素
移动

任何情况下：都有做 $n-1$ 趟



10.4 选择排序

10.4.1 简单选择排序-算法分析

- 从 i 个元素中挑选最小元素需要比较 $i-1$ 次。
- 第 i ($i=0 \sim n-2$) 趟从 $n-i$ 元素中挑选最小元素需要比较 $n-i-1$ 次。

对 n 个元素进行简单选择排序，所需进行的关键字的比较次数总计为：

$$\sum_{i=0}^{n-2} (n-i-1) = \frac{n(n-1)}{2}$$

- 移动元素的次数，正序为最小值 0，反序为最大值 $3(n-1)$ 。

简单选择排序的最好、最坏和平均时间复杂度为 $O(n^2)$ 。

10.4 选择排序

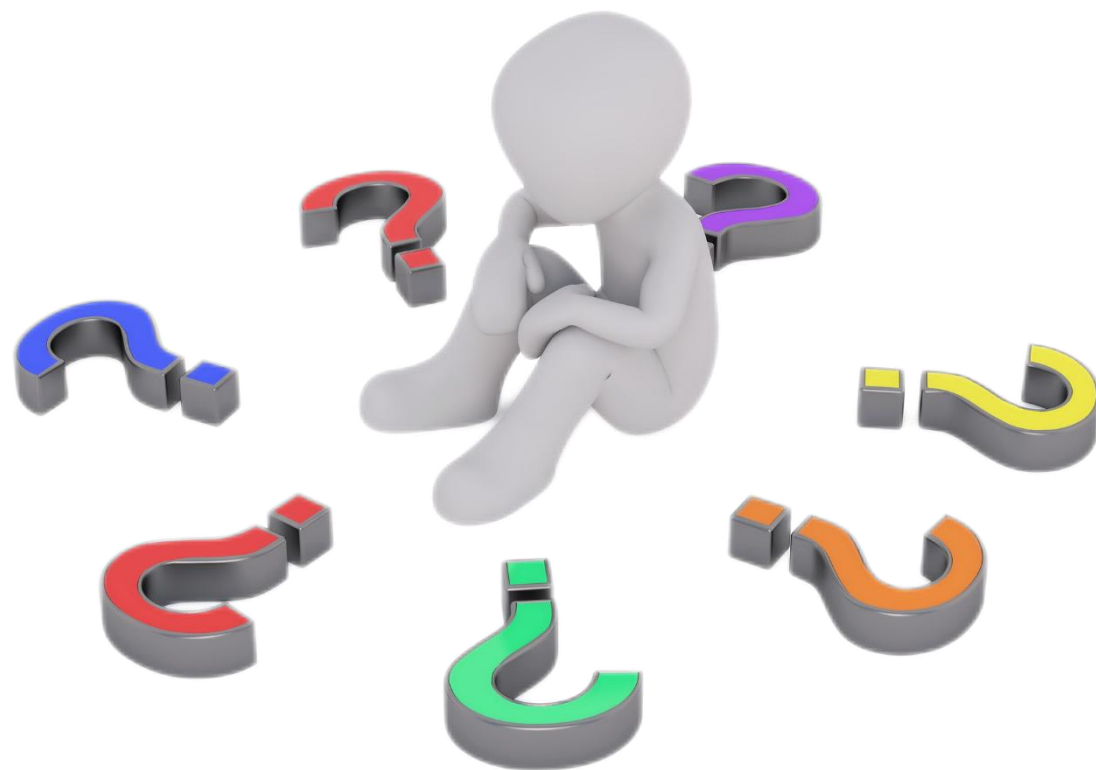
10.4.1 简单选择排序-算法分析

阿里巴巴的面试题

在 2^N 个整数中找到一个最大的整数，需要的比较次数是（ ）。

- A. N
- B. 2^N
- C. $2^N - 1$
- D. $\log_2 N$
- E. $2N$
- F. $2N - 1$

C



设有一组初始关键字序列为(24, 35, 12, 27, 18, 26), 则每一趟简单选择排序结束后的结果的是? 采用冒泡排序法, 每一趟排序结果是?
从小到大排序, 冒泡从右往左冒

简单选择排序

第1趟: (12, 35, 24, 27, 18, 26)
第2趟: (12, 18, 24, 27, 35, 26)
第3趟: (12, 18, 24, 27, 35, 26)
第4趟: (12, 18, 24, 26, 35, 27)
第5趟: (12, 18, 24, 26, 27, 35)

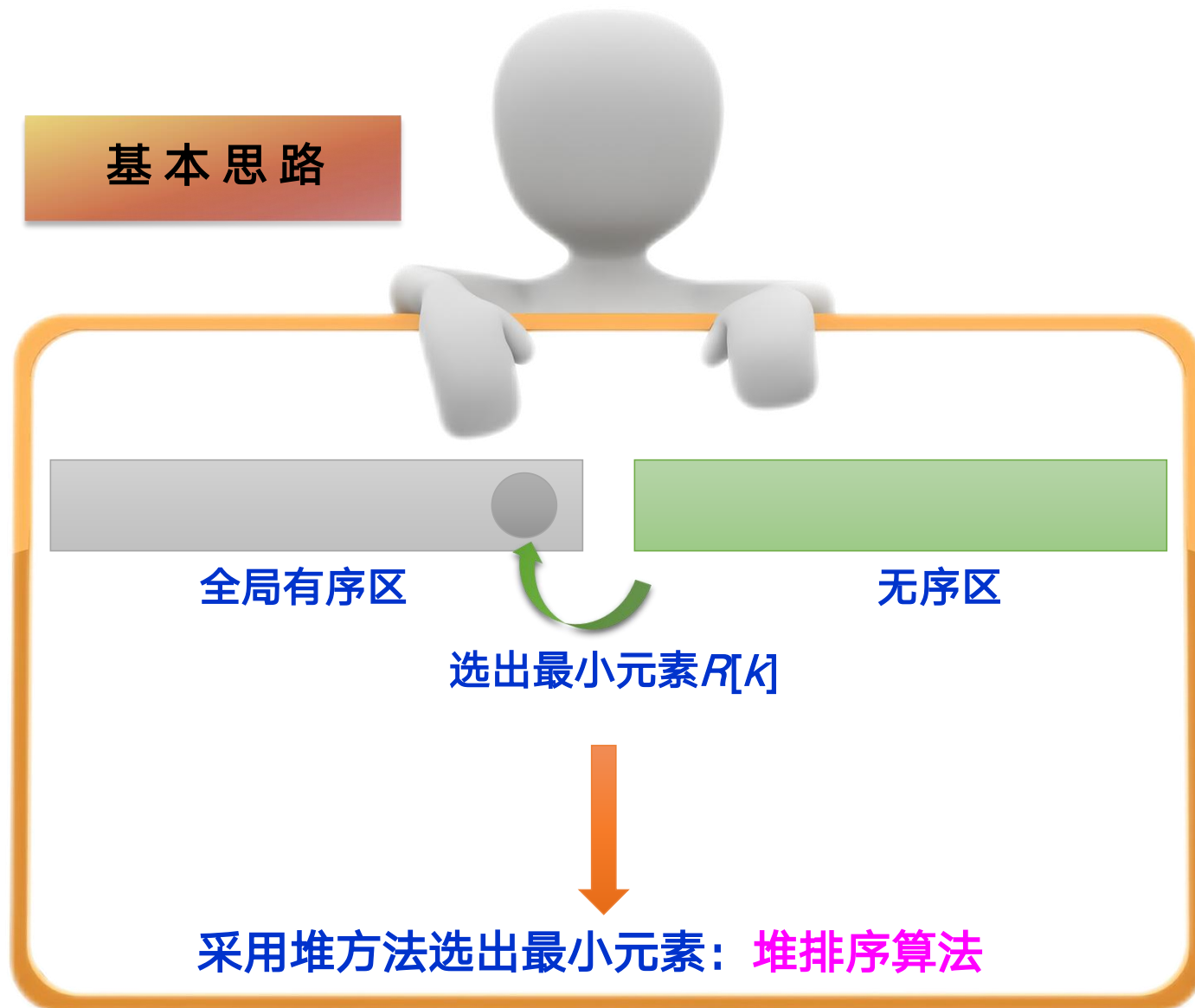
冒泡排序法

第1趟: (12, 24, 35, 18, 27, 26)
第2趟: (12, 18, 24, 35, 26, 27)
第3趟: (12, 18, 24, 26, 35, 27)
第4趟: (12, 18, 24, 26, 27, 35)
第5趟: (12, 18, 24, 26, 27, 35)

10.4 选择排序

10.4.2 堆排序

基本思路



10.4 选择排序

10.4.2 堆排序

堆的定义

一个序列 $R[1..n]$ ，关键字分别为 $k_1、k_2、\dots、k_n$ 。

该序列满足如下性质（简称为堆性质）：

① $k_i \leq k_{2i}$ 且 $k_i \leq k_{2i+1}$

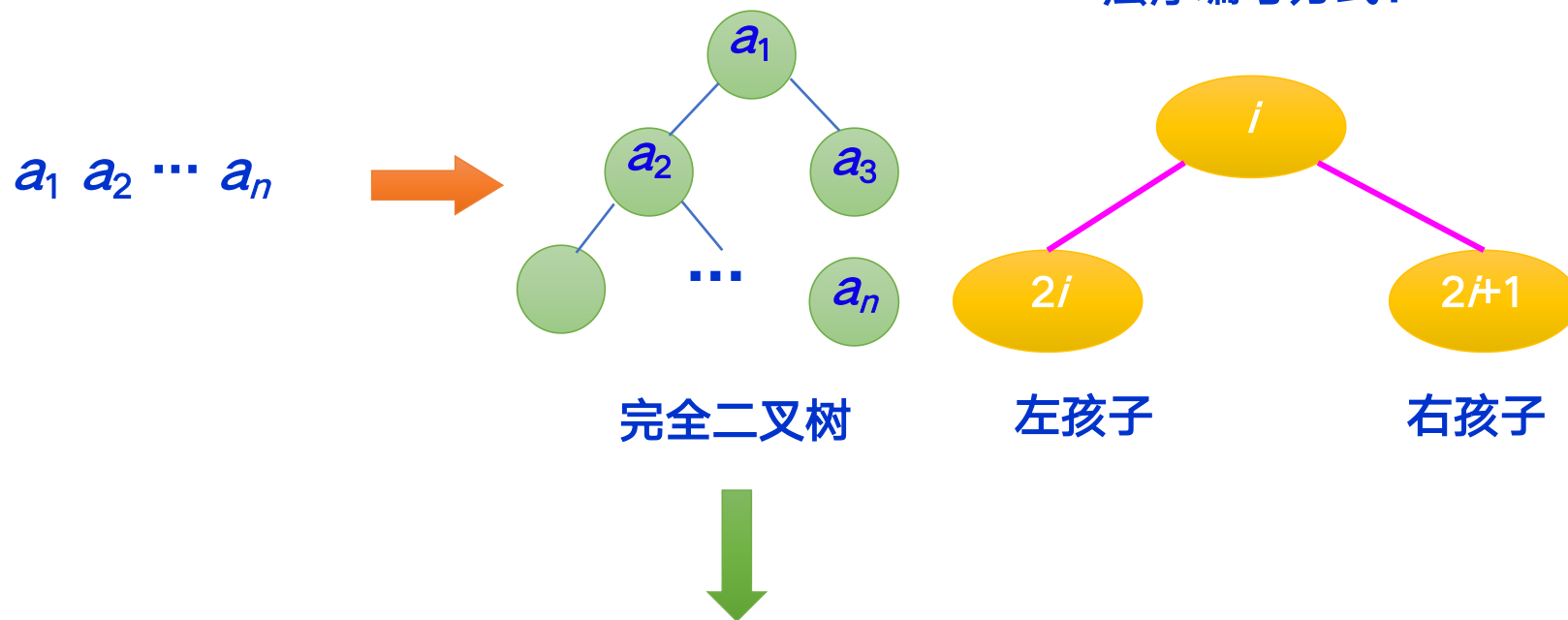
或 ② $k_i \geq k_{2i}$ 且 $k_i \geq k_{2i+1}$ ($1 \leq i \leq \lfloor n/2 \rfloor$)

满足第①种情况的堆称为**小根堆**，满足第②种情况的堆称为**大根堆**。下面讨论的堆是**大根堆**。

10.4 选择排序

10.4.2 堆排序

❶ 将序列 $a_1 a_2 \cdots a_n$ 看成是一颗完全二叉树



- **大根堆**：对应的完全二叉树中，任意一个结点的关键字都大于或等于它的孩子结点的关键字。
- 最小关键字的元素一定是某个叶子结点！！！！

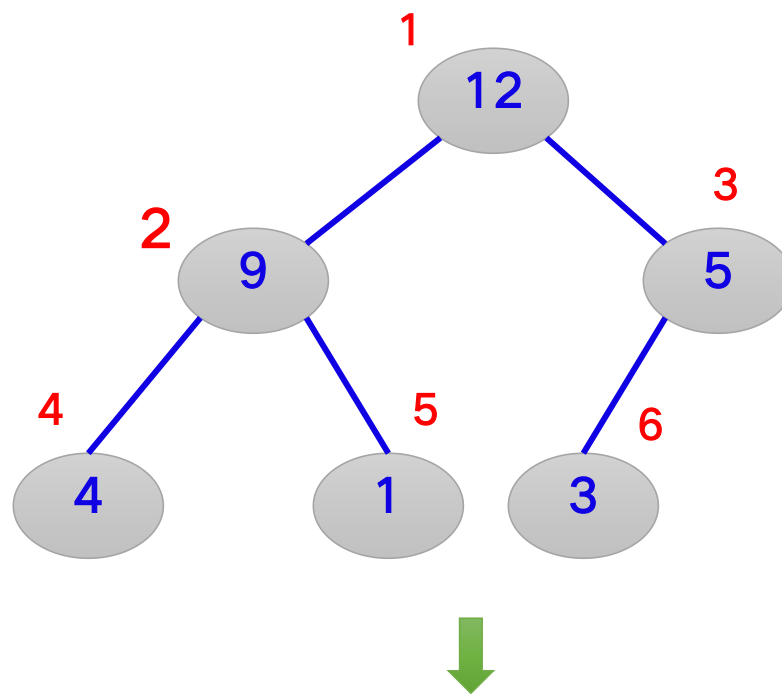
10.4 选择排序

10.4.2 堆排序

② 如何判断一颗完全二叉树是否为大根堆

$n=6$

从最后一个分支结点开始判断，
编号为 $n/2=3$ 的结点开始，逐一判断
所有分支结点



所有分支结点满足定义 $\Rightarrow$ 大根堆

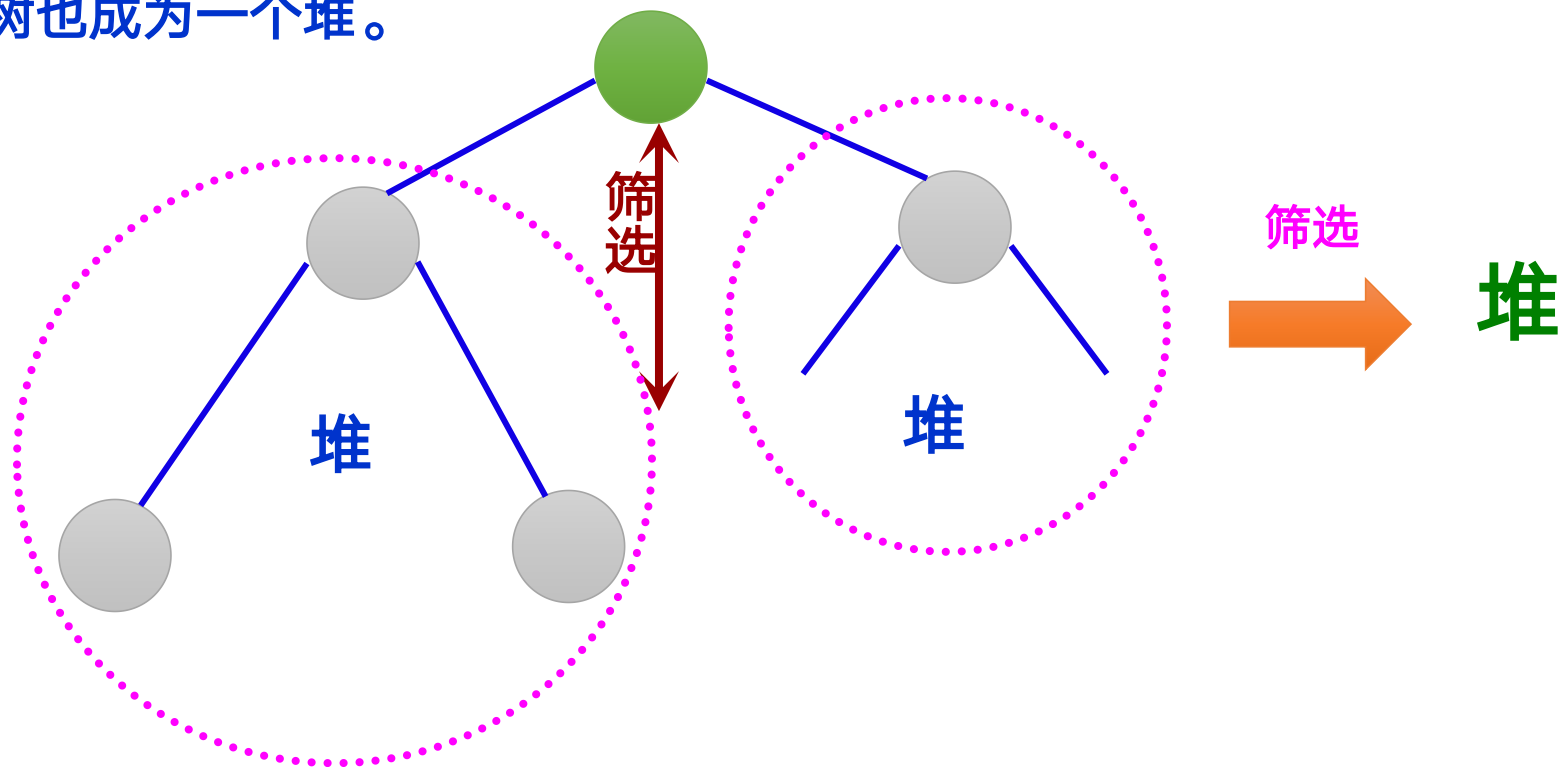
10.4 选择排序

10.4.2 堆排序

堆排序算法设计

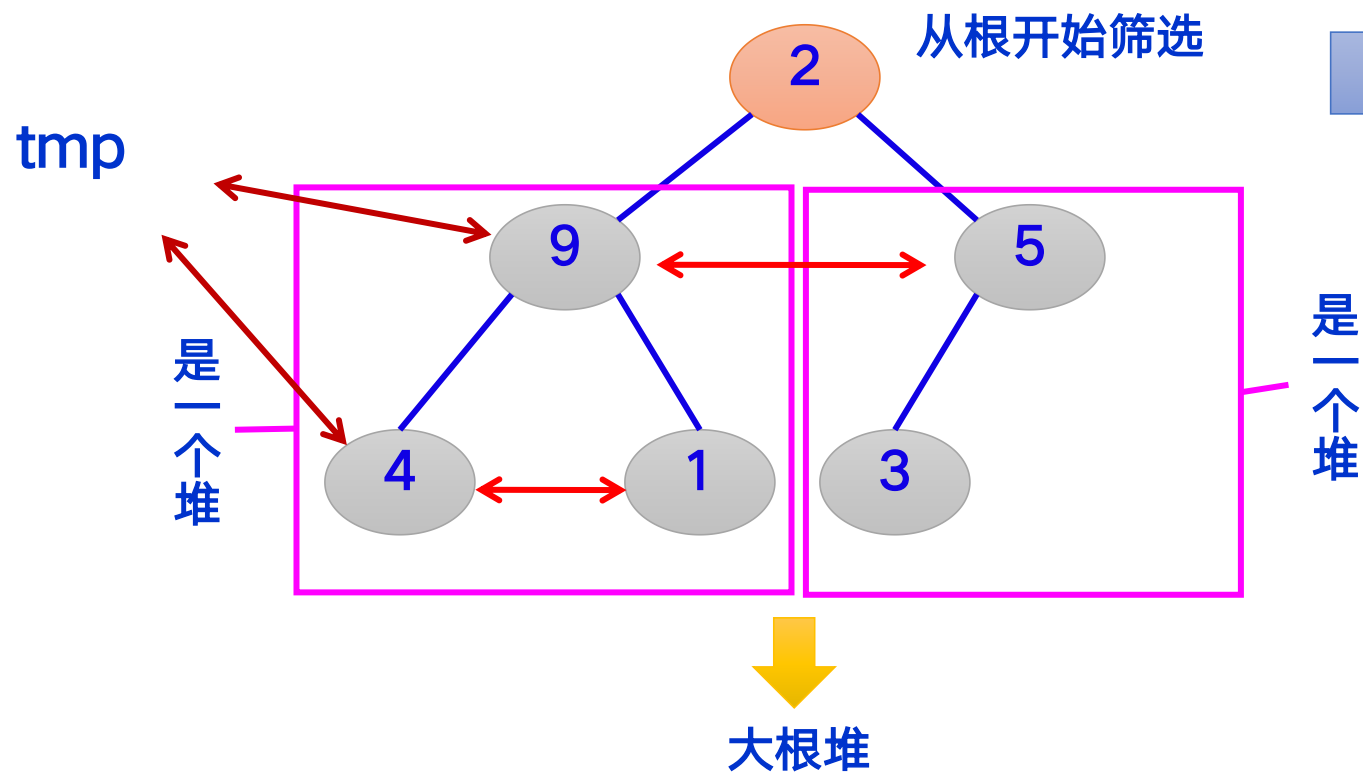
堆排序的关键是构造堆，这里采用**筛选算法**建堆。

所谓“**筛选**”指的是，对一棵左/右子树均为堆的完全二叉树，“调整”根结点使整个二叉树也成为堆。



10.4 选择排序

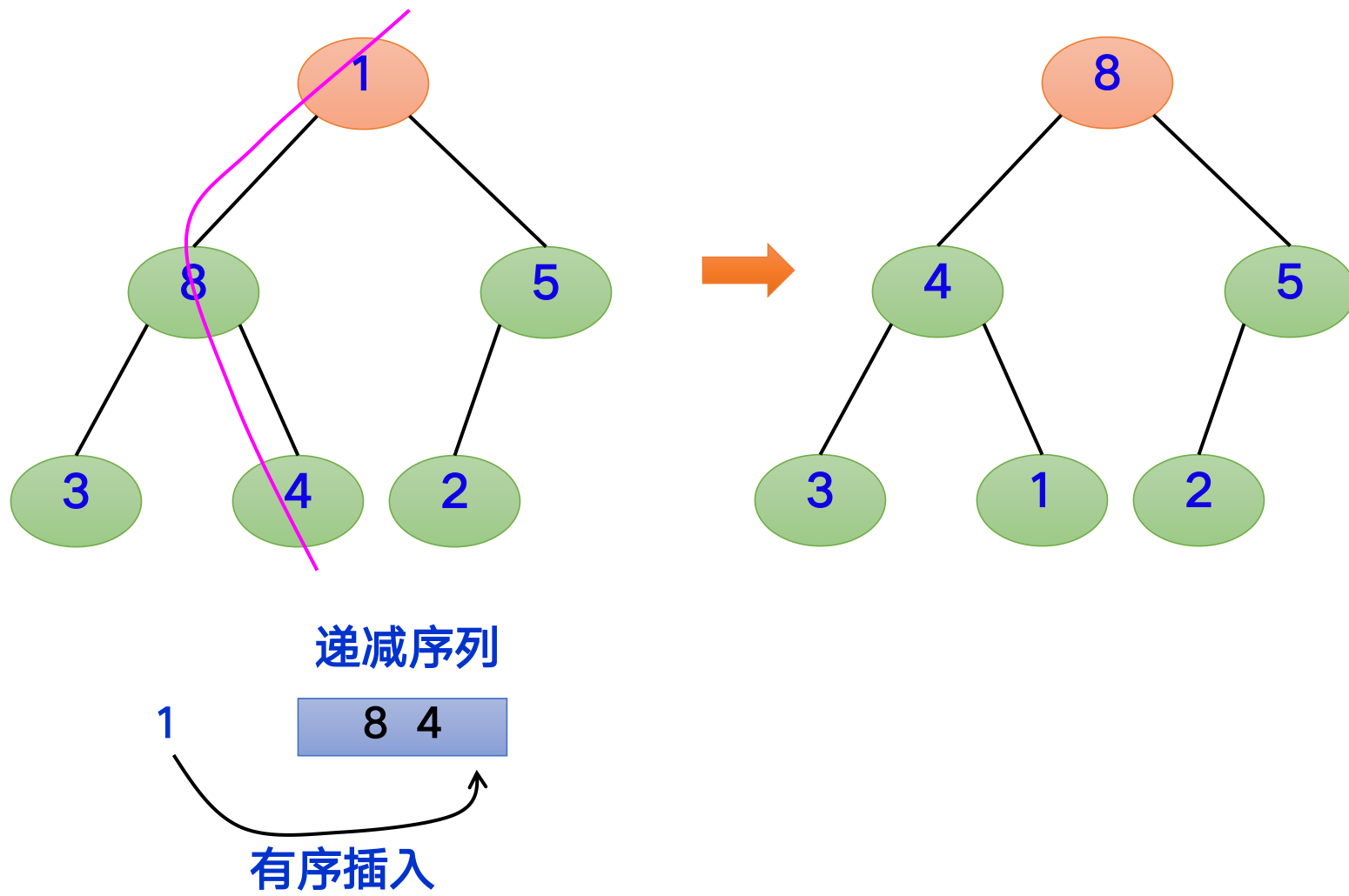
10.4.2 堆排序



- 仅仅处理从根结点 $\Rightarrow$ 某个叶子结点路径上的结点
- n 个结点的完全二叉树高度为 $\lceil \log_2(n+1) \rceil$
- 所有筛选的时间复杂度为 $O(\log_2 n)$

10.4 选择排序

10.4.2 堆排序

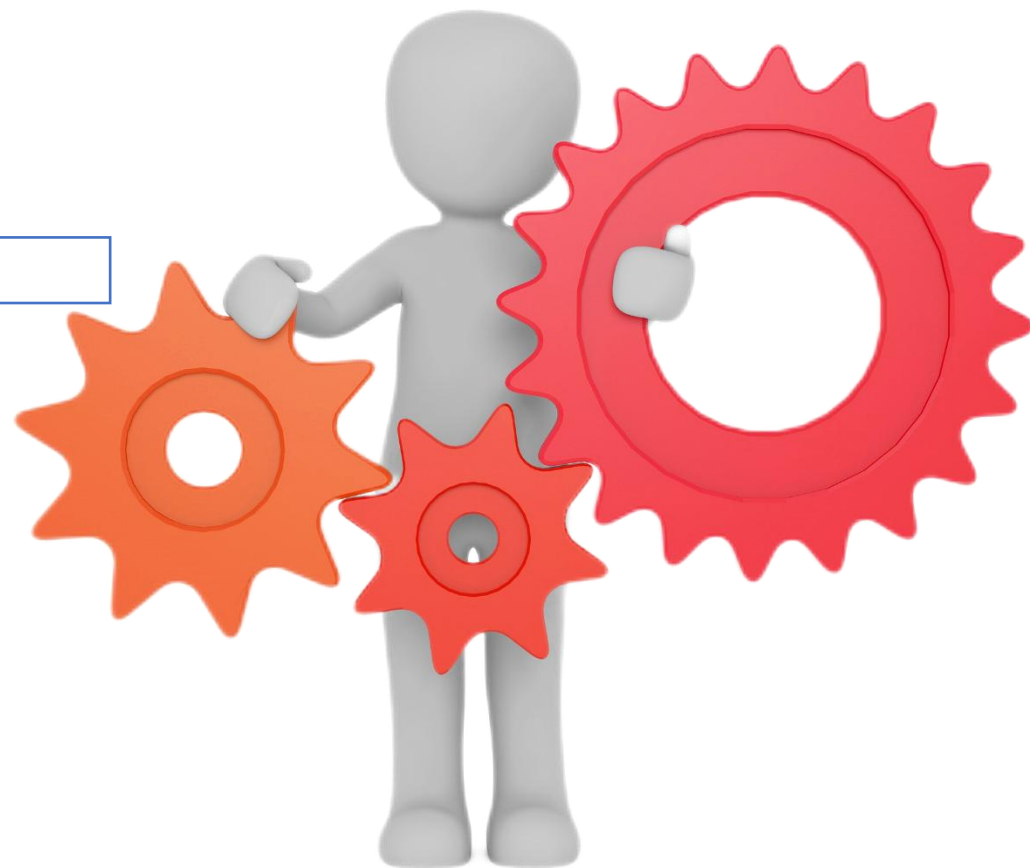
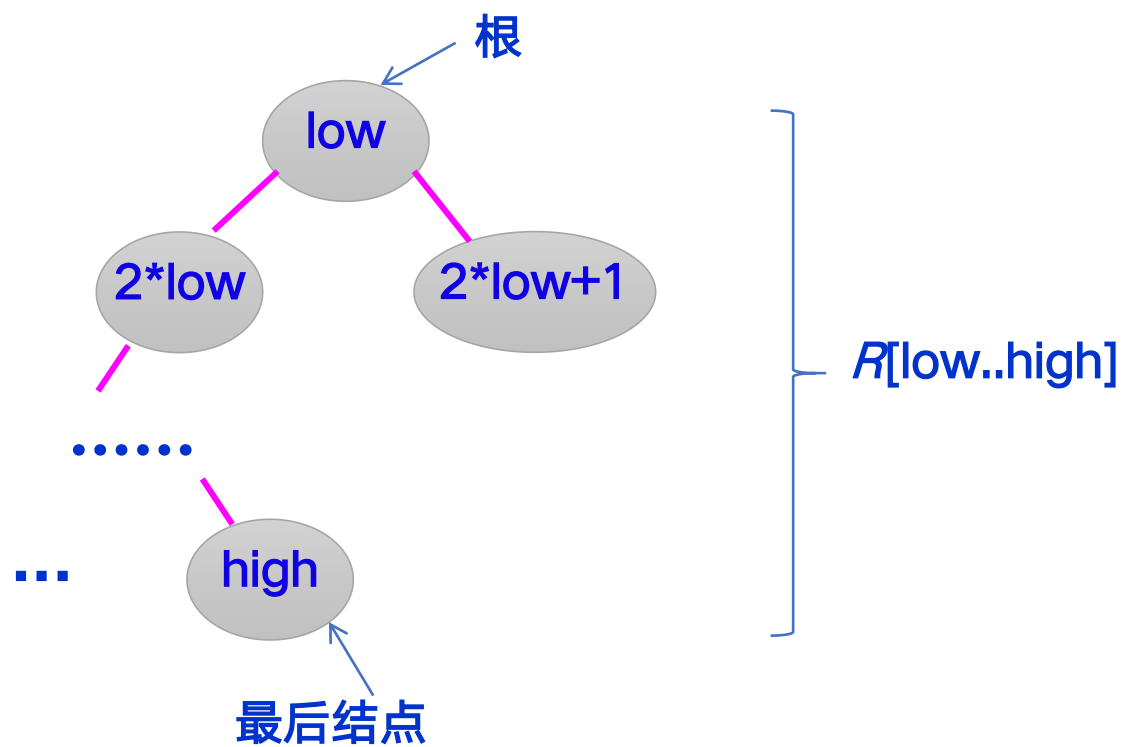


10.4 选择排序

10.4.2 堆排序

筛选算法

`sift(RecType R[], int low, int high):` $R[\text{low}..\text{high}]$



10.4 选择排序

10.4.2 堆排序

筛选或调整算法

```
void sift(RecType R[], int low, int high) //调整堆的算法
{ int i=low, j=2*i;           //R[j]是R[i]的左孩子
  RecType tmp=R[i];
  while (j<=high)
  {
    if (j<high && R[j].key<R[j+1].key) j++;
    if (tmp.key<R[j].key) //双亲小
    { R[i]=R[j];          //将R[j]调整到双亲结点位置
      i=j;                //修改i和j值，以便继续向下筛选
      j=2*i;
    }
    else break;           //双亲大：不再调整
  }
  R[i]=tmp;
}
```

指向
大孩子

10.4 选择排序

10.4.2 堆排序

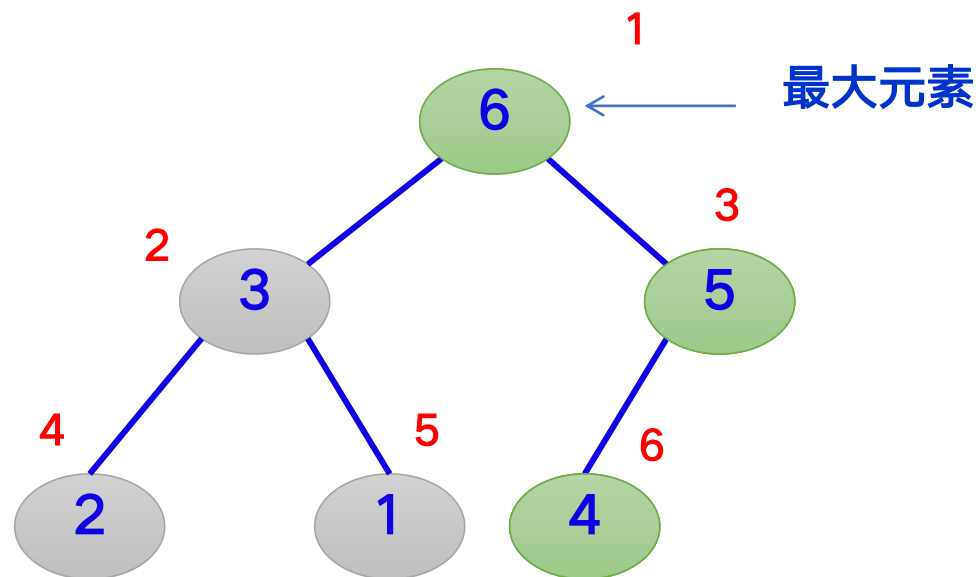
② 一颗完全二叉树 $\Rightarrow$ 初始堆

例如，序列：(4, 3, 5, 2, 1, 6)，

$n=6$

从编号为 $n/2=3$ 的
结点开始，逐一筛
选

```
for (i=n/2;i>=1;i--) //循环建立初始堆  
    sift(R, i, n);
```



筛选步骤：

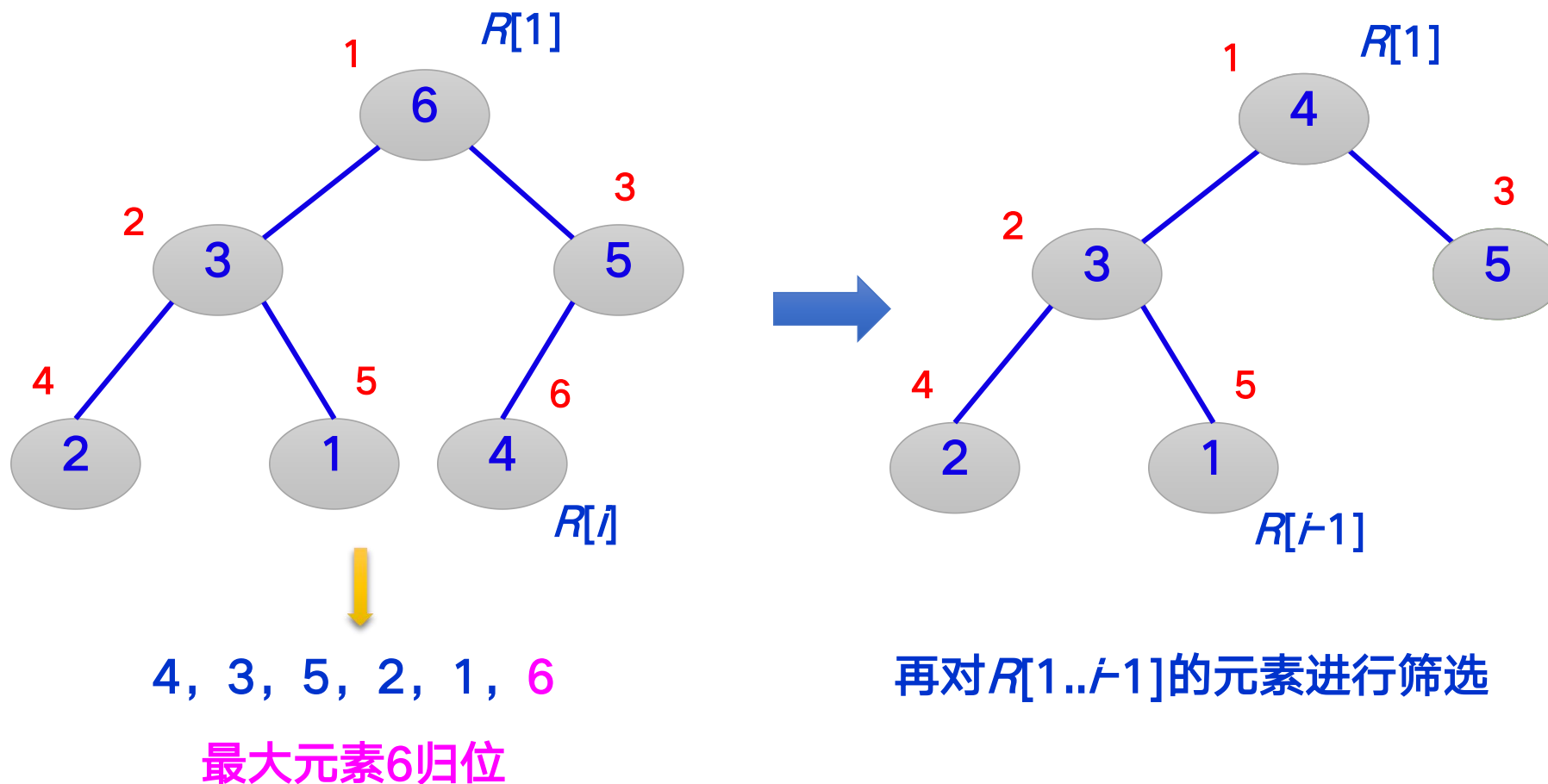
- ① $\text{sift}(R, 3, 6)$
- ② $\text{sift}(R, 2, 6)$
- ③ $\text{sift}(R, 1, 6)$

初始堆: (6, 3, 5, 2, 1, 4)

10.4 选择排序

10.4.2 堆排序

③ 最大元素归位



10.4 选择排序

10.4.2 堆排序

堆排序算法

```
void HeapSort(RecType R[], int n)
{
    int i;
    for (i=n/2;i>=1;i--)           //循环建立初始堆
        sift(R, i, n);

    for (i=n; i>=2; i--)           //进行n-1次循环，完成堆排序
    {
        swap(R[1],R[i]);           //R[1]  $\leftrightarrow$  R[i]
        sift(R, 1, i-1);           //筛选R[1]结点，得到i-1个结点的堆
    }
}
```

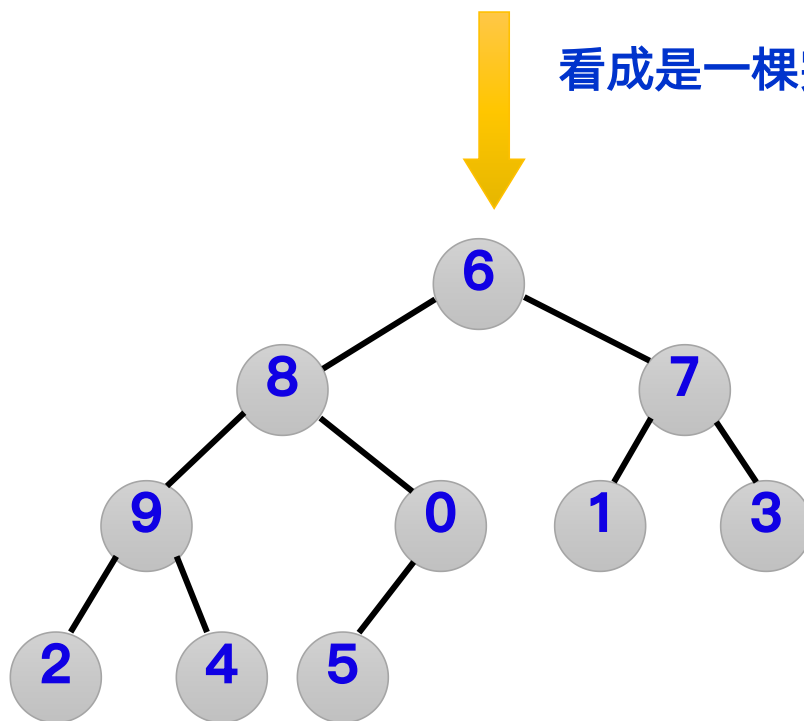
10.4 选择排序

10.4.2 堆排序

【例10.6】 设待排序的表有10个元素，其关键字分别为{6, 8, 7, 9, 0, 1, 3, 2, 4, 5}。
说明采用堆排序方法进行排序的过程。

排序序列：6, 8, 7, 9, 0, 1, 3, 2, 4, 5

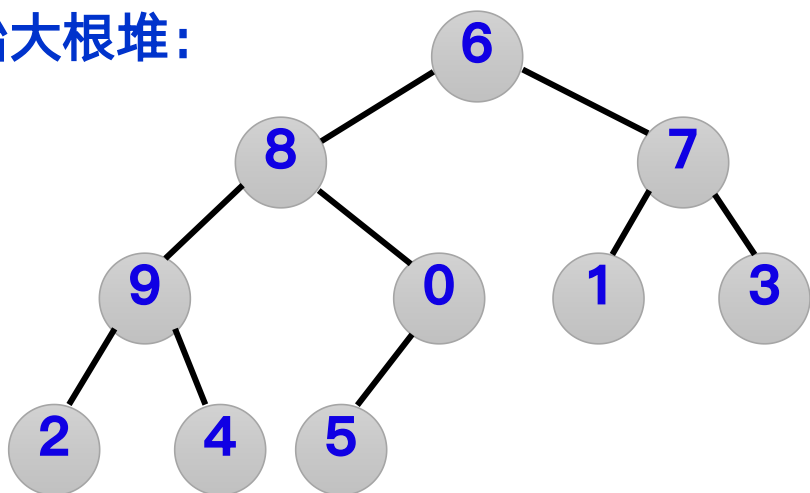
看成是一棵完全二叉树



10.4 选择排序

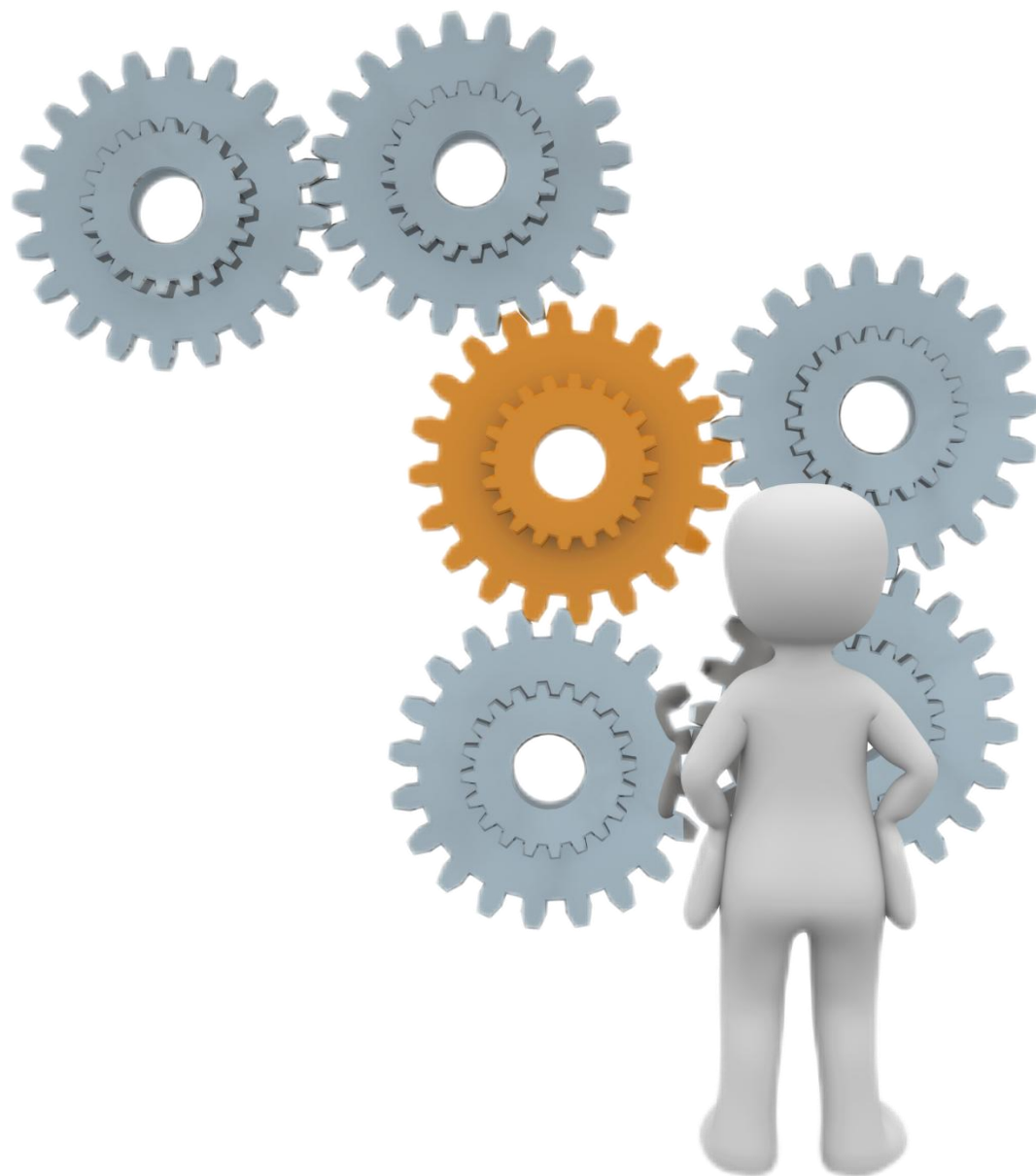
10.4.2 堆排序

调整成初始大根堆:



调整完毕，成为一个大根堆

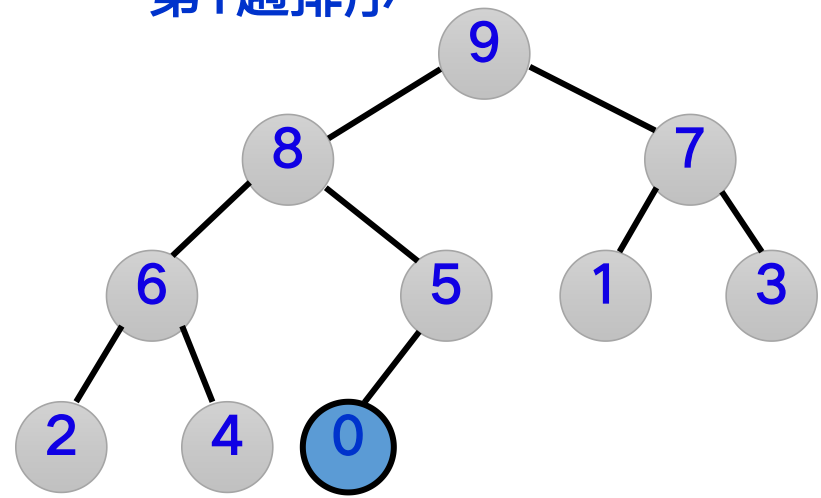
9 8 7 6 5 1 3 2 4 0



10.4 选择排序

10.4.2 堆排序

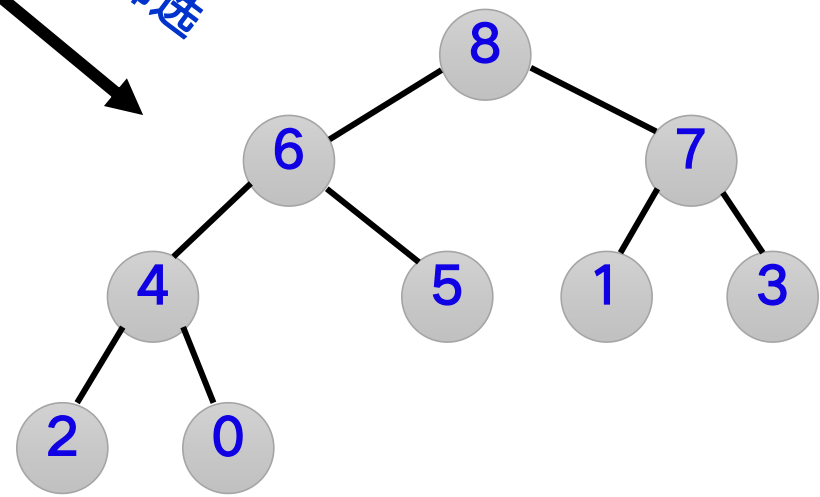
第1趟排序



0 8 7 6 5 1 3 2 4 9

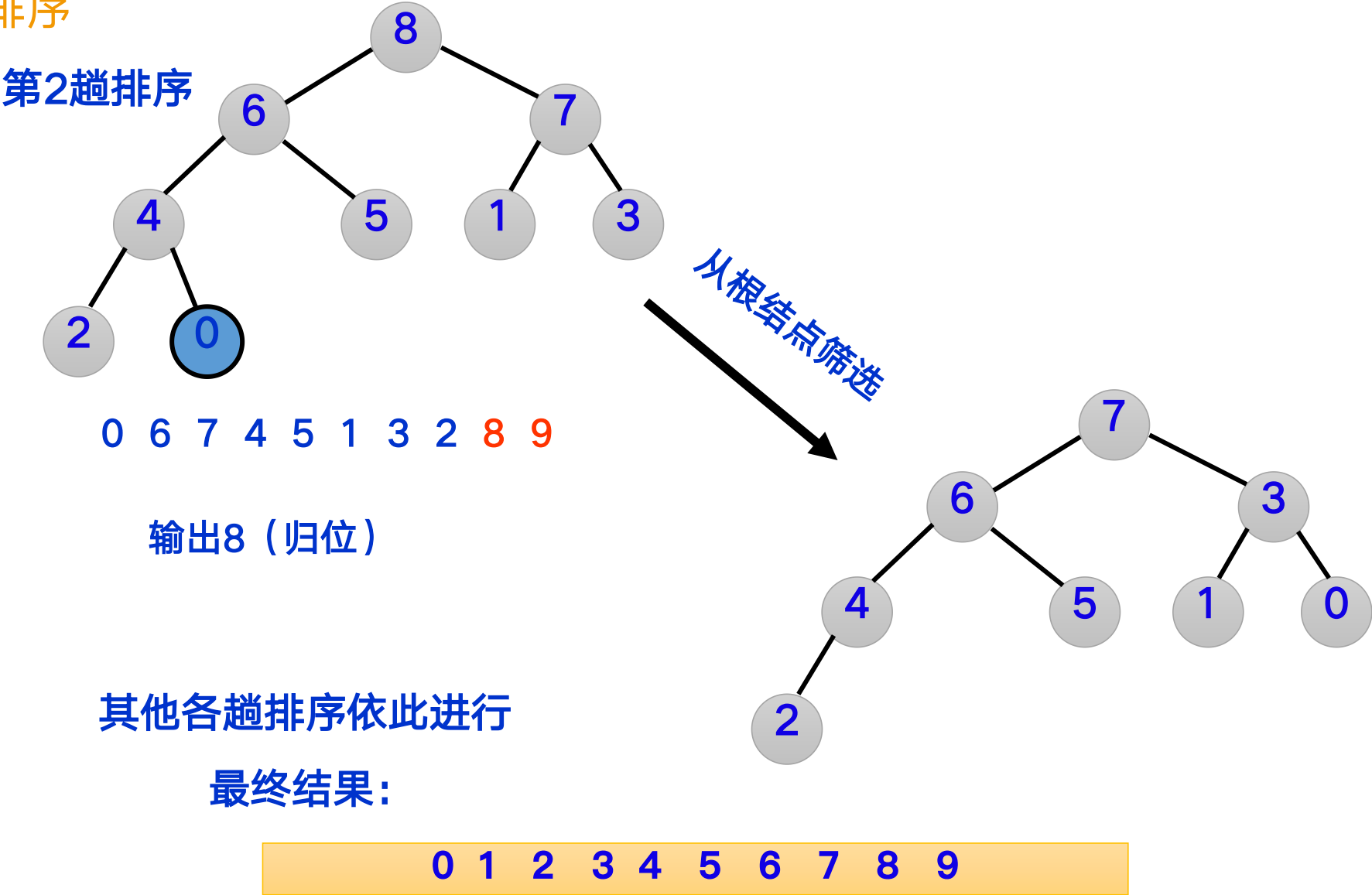
输出9（归位）

从根结点筛选



10.4 选择排序

10.4.2 堆排序



10.4 选择排序

10.4.2 堆排序

数据结构经典算法的启示

简单选择排序算法



利用了连续多次查找最大元素的特性

堆排序算法

在操作系统中，将多个进程放在一个队列中，每个进程有一个优先级，总是出队优先级最高的进程执行。

采用**优先队列**，用**堆**来实现！



2018统考真题：在将数据序列 (6, 1, 5, 9, 8, 4, 7) 建成大根堆时，正确的序列变化过程是 ()

- A. 6, 1, 7, 9, 8, 4, 5 ; 6, 9, 7, 1, 8, 4, 5 ; 9, 6, 7, 1, 8, 4, 5 ; 9, 8, 7, 1, 6, 4, 5
- B. 6, 9, 5, 1, 8, 4, 7 ; 6, 9, 7, 1, 8, 4, 5 ; 9, 6, 7, 1, 8, 4, 5 ; 9, 8, 7, 1, 6, 4, 5
- C. 6, 9, 5, 1, 8, 4, 7 ; 9, 6, 5, 1, 8, 4, 7 ; 9, 6, 7, 1, 8, 4, 5 ; 9, 8, 7, 1, 6, 4, 5
- D. 以上均不正确

答案：A

10.7 各种内排序的比较

各种内排序方法的性能

排序方法	时间复杂度			空间复杂度	稳定性
	平均情况	最坏情况	最好情况		
直接插入排序	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$	稳定
折半插入排序	$O(n^2)$	$O(n^2)$	$O(n\log_2 n)$	$O(1)$	稳定
希尔排序	$O(n^{1.3})$			$O(1)$	不稳定
冒泡排序	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$	稳定
快速排序	$O(n\log_2 n)$	$O(n^2)$	$O(n\log_2 n)$	$O(\log_2 n)$	不稳定
简单选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定
堆排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(1)$	不稳定
二路归并排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n)$	稳定
基数排序	$O(d(n+r))$	$O(d(n+r))$	$O(d(n+r))$	$O(r)$	稳定